REDUCING HALLUCINATIONS IN LEGAL AI: RETRIEVAL-AUGMENTED GENERATION WITH A GATED VERIFICATION CHAIN

A Mini Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

by

Nalla Akshith Sai (23EG106C03)
Aleti Vivek Reddy (23EG106C04)
Kollu Uday (23EG106C30)

Under the guidance of

Mr. S. Ravi Kumar
Assistant Professor,
Department of Artificial Intelligence



DEPARTMENT OF ARTIFICIAL INTELLIGENCE
ANURAG UNIVERSITY
Venkatapur, Ghatkesar, Hyderabad - 500088

October, 2026

DECLARATION

We hereby declare that the Mini Project report entitled "**REDUCING HALLUCINATIONS IN LEGAL AI: RETRIEVAL-AUGMENTED GENERATION WITH A GATED VERIFICATION CHAIN**" submitted to the Department of Artificial Intelligence, Anurag University, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, is a record of original work carried out by us under the guidance of Mr. S. Ravi Kumar, Assistant Professor, Department of Artificial Intelligence. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma. Wherever the work of others, including pretrained models, open source libraries and published papers, has been used, it has been duly acknowledged and cited.

Use of AI tools: We declare that we have used large language model tools for code development and debugging assistance, language editing and drafting support while preparing this project and report. The problem definition, design decisions, testing and interpretation of results are our own, and we take full responsibility for the content of this report.

Place: Hyderabad
Date: _____

Name of the Student	Roll No.	Signature
Nalla Akshith Sai	23EG106C03	
Aleti Vivek Reddy	23EG106C04	
Kollu Uday	23EG106C30	

CERTIFICATE

This is to certify that the Mini Project report entitled "**REDUCING HALLUCINATIONS IN LEGAL AI: RETRIEVAL-AUGMENTED GENERATION WITH A GATED VERIFICATION CHAIN**" is a bonafide record of the work carried out by Nalla Akshith Sai (23EG106C03), Aleti Vivek Reddy (23EG106C04) and Kollu Uday (23EG106C30) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence during the academic year 2026 to 2027. The work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Mr. S. Ravi Kumar
Project Guide,
Assistant Professor,
Department of Artificial Intelligence

Prof. M. Trupthi,
Head of the Department
Department of Artificial Intelligence

ACKNOWLEDGEMENTS

We express our sincere gratitude to our guide, Mr. S. Ravi Kumar, Assistant Professor, Department of Artificial Intelligence, for his valuable guidance, constant encouragement and support throughout this project. His feedback at every review helped us shape the scope of Legal AI, keep the system design simple and practical, and report our work honestly.

We thank Dr. M. Trupthi, Head of the Department of Artificial Intelligence, for providing the facilities and the academic environment needed to carry out this project, and for her support during the project reviews.

We thank the Project Review Committee members of the Department of Artificial Intelligence for their suggestions during the reviews, which helped us improve the system design and the evaluation plan.

We are grateful to Prof. V. Vijaya Kumar, Dean, School of Engineering, Dr. Archana Mantri, Vice Chancellor, and the Management of Anurag University for providing the computing facilities, library resources, and academic environment necessary for successfully completing this work.

We also thank all the faculty and staff of the Department of Artificial Intelligence, our families and our friends for their continued support and encouragement.

Nalla Akshith Sai (23EG106C03)
Aleti Vivek Reddy (23EG106C04)
Kollu Uday (23EG106C30)

ABSTRACT

Large language models are increasingly used to answer legal questions, and they sometimes state rules or cite sources that do not exist. In a well-known case, a court sanctioned lawyers who had filed cases invented by a chatbot, and evaluations of commercial legal research tools show that even systems built on document retrieval still produce such errors. Retrieval-augmented generation reduces the problem by giving the model passages from trusted documents, but it does not check the answer that the model finally writes. A wrong claim with a plausible citation therefore looks exactly like a correct one to the user. This project addresses that gap with Legal AI, a legal question-answering system that splits statutes by section with summary-augmented chunking, finds evidence with dense and BM25 keyword search merged by reciprocal rank fusion and reranked by a cross-encoder, and requires the language model to cite a source passage for every claim.

A six-layer verification chain checks every claim against its cited passage with an inference model, an atomic fact check and an optional consistency check, and reduces these checks to a Verification Confidence Score; the answer is shown only when the score is at least 0.60, otherwise the system abstains, and every answer carries a proof that the user can open. The system is delivered as a FastAPI and React web application with a demonstration pack of Indian criminal law, and all its code is on GitHub. It was evaluated with ablation experiments and an end-to-end test through its web interface. On a labelled set of ten questions over five synthetic statutes, summary-augmented chunking raised retrieval F1 from 0.333 to 0.417 over paragraph chunking while passing about a third as much text to the generator. Of twenty-eight deliberately corrupted claims, the full chain showed none to the user; removing any single detection layer still showed none, while removing the score gate showed all twenty-eight. In an end-to-end test with five real documents and fifteen questions whose answers were fixed in advance, no wrong answer was shown as verified, while the main cost of this caution was false abstention.

Keywords: legal AI; retrieval-augmented generation; hallucination; citation verification; natural language inference; abstention; LanceDB

TABLE OF CONTENTS

Abstract i
List of Figures v
List of Tables vi
List of Abbreviations vii

1 Introduction 1
1.1 Background 1
1.1.1 Overview of Legal AI 1
1.1.2 Key Terms in Retrieval-Augmented Generation 2
1.1.3 Why Legal Question Answering Is Difficult 3
1.2 Motivation 4
1.3 Problem Statement 4
1.4 Research Objectives 5
1.5 Scope and Limitations 7
1.5.1 Assumptions 7
1.5.2 Outside the Scope 7
1.6 Organization of the Report 8

2 Literature Survey 9
2.1 Review of Related Work 9
2.1.1 Large Language Models and Hallucination 9
2.1.2 Retrieval-Augmented Generation 10
2.1.3 Chunking Strategies for Legal Documents 10
2.1.4 Hybrid Retrieval and Reranking 11
2.1.5 Verification of Generated Answers 11
2.1.6 Legal Datasets and Benchmarks 12
2.1.7 Positioning of the Present Work 13
2.2 Summary of Literature 13
2.3 Research Gap 14

3 System Analysis and Requirements 16
3.1 Existing Approaches and their Drawbacks 16
3.2 Proposed System 17
3.3 Feasibility Study 17
3.4 Hardware Requirements 18
3.5 Software Requirements 18
3.5.1 Functional Requirements 19
3.5.2 Non Functional Requirements 20
3.5.3 Development Environment 20

4 System Design and Methodology 22
4.1 Overall Pipeline 22
4.2 System Architecture 23
4.3 Module Description 24
4.3.1 Ingestion and Duplicate Detection Module 24
4.3.2 Summary-Augmented Chunking Module 25

4.3.3 Embedding and Indexing Module 25
4.3.4 Hybrid Retrieval and Reranking Module 25
4.3.5 Citation-Forced Generation Module 26
4.3.6 Verification Module 26
4.3.7 Web Interface Module 27
4.4 Model Selection and Justification 27
4.5 Methodology 28
4.5.1 Section-Aware Chunking Algorithm 28
4.5.2 Reciprocal Rank Fusion 29
4.5.3 Claim Score and Verification Confidence Score 29
4.5.4 Abstention and General-Knowledge Routing 30
4.6 Data Design 30
4.7 UML Diagrams 31
4.7.1 Use Case Diagram 31
4.7.2 Sequence Diagram 32
4.7.3 Activity Diagram 33
4.7.4 Class Diagram 34
4.7.5 Data Flow Diagrams 35
4.8 Comparison with Existing Approaches 36

5 Implementation 38
5.1 Experimental Setup 38
5.2 Project Structure 38
5.3 Datasets Used 39
5.4 Key Parameters 40
5.5 Key Code Snippets 41
5.5.1 Text Extraction 41
5.5.2 Query Pipeline 43
5.5.3 Retrieval Comparison and Fusion 45
5.5.4 Verification Rules and Orchestration 47
5.6 Deployment and User Interface 48

6 Results and Discussion 50
6.1 Evaluation Metrics 50
6.2 Testing Strategy 50
6.3 Functional Test Results 51
6.4 Quantitative Results 52
6.4.1 Retrieval and Chunking 52
6.4.2 Verification Ablation 54
6.4.3 End-to-End Test Through the Interface 55
6.4.4 LangChain and Plain-Python Comparison 56
6.5 Comparison with Existing Methods 57
6.6 Error Analysis 57
6.7 Sample Outputs 58
6.8 Ethical and Responsible AI Considerations 60

7 Conclusion and Future Scope 62
7.1 Conclusion 62

7.2 Limitations 62
7.3 Future Enhancements 63

References **64**

Appendices **67**
Appendix A: Source Code and Repository 67
Appendix B: Running and Hosting the System 67
Appendix C: Demonstration Questions 68

LIST OF FIGURES

Fig. No.	Title	Page No.
1.1	A verified answer in the Ask page of Legal AI, with its score and clickable citations	2
4.1	Processing pipeline of Legal AI, from document upload to the decision to answer or abstain	22
4.2	Three-tier system architecture of Legal AI	23
4.3	The verification chain: V1 to V4 score each claim, V5 turns the scores into one decision and V6 records the evidence	27
4.4	Use case diagram of Legal AI	32
4.5	Sequence diagram for answering a question	33
4.6	Activity diagram of the verification chain and the decision rule	34
4.7	Class diagram of the main components	35
4.8	Level 0 data flow diagram (context diagram)	36
4.9	Level 1 data flow diagram	36
5.1	The Ingest page, where documents are uploaded and indexed	48
5.2	The Search page, comparing dense, keyword and hybrid retrieval for one query	49
6.1	Retrieval F1 for summary-augmented and paragraph chunking under each retrieval setting	53
6.2	Corrupted claims shown to the user when each layer is removed, out of 28; only removing the V5 gate lets them through	54
6.3	A verified but incomplete answer: two correct claims for a question about three agreements; the note about the missing third had no citation and was dropped	58
6.4	Proof view of a verified answer on a real non-disclosure agreement, with the cited paragraph and the verdict of each layer	59
6.5	An abstention: no indexed agreement covers repeat unfair trade practice offences, so no claim is shown	60
6.6	The Evaluation page's map of the index: 695 chunks from eight documents, projected with PCA and coloured by document	60

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of Reviewed Literature	13
3.1	Feasibility Study Summary	18
3.2	Hardware Requirements	18
3.3	Software Requirements	19
4.1	Models Used and Justification	28
4.2	Structure of a Chunk Row in the LanceDB Index	30
4.3	Design Comparison with Existing Approaches	37
5.1	Experimental and Development Setup	38
5.2	Main Folders of the Project Repository	39
5.3	Key Implementation Parameters	40
6.1	Functional Test Cases	51
6.2	Retrieval Results over 10 Questions (k = 5, n = 50); Ctx is the Mean Characters of Context per Question	53
6.3	Verification Ablation: Clean Set of 10 Questions and 14 Claims; 14 Fabricated and 14 Swapped Claims	54
6.4	End-to-End Test: 15 Questions with Answers Fixed in Advance	55
6.5	Plain-Python and LangChain Implementations Compared	56
6.6	Sample System Outputs	59
C.1	Selected Demonstration Questions and Expected Behaviour	68

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BM25	Best Matching 25 (keyword ranking function)
CPU	Central Processing Unit
CrPC	Code of Criminal Procedure
CUAD	Contract Understanding Atticus Dataset
DFD	Data Flow Diagram
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IPC	Indian Penal Code
JSON	JavaScript Object Notation
LLM	Large Language Model
MAUD	Merger Agreement Understanding Dataset
MNLI	Multi-Genre Natural Language Inference
NI Act	Negotiable Instruments Act
NLI	Natural Language Inference
PCA	Principal Component Analysis
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer
RoBERTa	Robustly Optimized BERT Pretraining Approach
RRF	Reciprocal Rank Fusion
SAC	Summary-Augmented Chunking
SHA-256	Secure Hash Algorithm, 256-bit
SNLI	Stanford Natural Language Inference
UI	User Interface
UML	Unified Modeling Language
VCS	Verification Confidence Score

CHAPTER 1
INTRODUCTION

1.1 Background

Large language models have changed the way people look for information. A student preparing for an examination, a small business owner reading a rental agreement and a junior lawyer drafting a notice can all type a question in plain words and receive a fluent answer within seconds. These models are built on the transformer architecture [1] and on pretrained language representations such as BERT [2], and they are trained on very large collections of text, which is why their answers sound confident and complete. In the legal domain this fluency is both useful and dangerous. Law is written in long, formal and heavily structured documents, and a short explanation in everyday language can save a reader a great deal of time. At the same time, a statement about the law is only valuable if it is correct and if the reader can trace it back to the provision that supports it.

The difficulty is that a language model does not look up the law when it answers. It predicts the next words from patterns it learned during training, and when those patterns are incomplete it can produce text that reads well but is not true. This behaviour is known as hallucination, and it has been studied across many natural language generation tasks [3]. In legal question answering it takes a particularly harmful form: the model may describe a section that does not exist, attach the wrong punishment to an offence, or cite a judgment that was never delivered. Studies of legal hallucination show that general-purpose models make such errors often, even on simple questions about well-known cases [4]. This project starts from that problem and asks how a question-answering system can be built so that its answers can be checked, and so that it refuses to answer when the evidence is not good enough.

1.1.1 Overview of Legal AI

Legal AI is a question-answering system for legal documents such as statutes and contracts. A user first uploads documents through a web interface. The system splits each document along its legal structure, so that every section or clause becomes a separate piece of text with its own identifier, and stores these pieces in a searchable index. When the user asks a question, the system finds the most relevant pieces, gives them to a large language model and instructs the model to write its answer one claim per line, with every claim ending in the identifier of the piece it came from. Before the answer is shown, every claim is checked against the passage it cites. The checks are combined into a single number, the Verification Confidence Score, and the answer is shown only when this score is at least 0.60; otherwise the system abstains and says that the documents do not support an answer.

The final answer is therefore not just a paragraph of text. It carries a badge that tells the user whether it was verified, a score, and a citation for each claim. Clicking a citation opens a proof view that shows the exact passage, the verdict of every verification layer and the contribution of the claim to the score. Fig. 1.1 shows a verified answer as it appears in the Ask page of the web interface. The system is built as a web application with a React front end and a FastAPI back end, and most of the models it uses run locally on the server. Only the step that writes the answer is sent to an external language model service, which is Groq running the open-weight model gpt-oss-120b in our deployment. Every other step, including the checking of the answer, is performed by components that we control and can inspect.

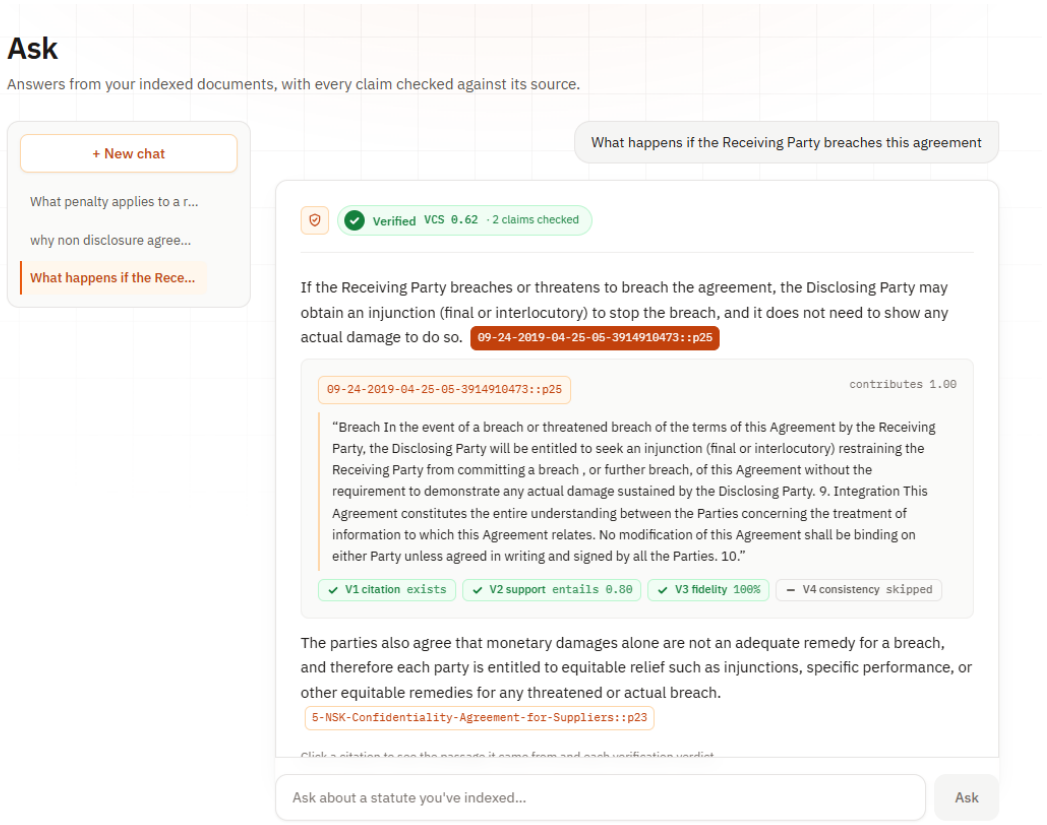


Fig. 1.1 A verified answer in the Ask page of Legal AI, with its score and clickable citations

1.1.2 Key Terms in Retrieval-Augmented Generation

Several technical terms are used throughout this report, and it is useful to explain them in one place. A large language model (LLM) is a neural network trained to predict text, which can answer questions and write explanations. Retrieval-augmented generation (RAG) is a design in which the system first retrieves relevant passages from a trusted collection of documents and then asks the model to answer from those passages instead of from memory alone [5]. A chunk is one of the small pieces into which a document is split before indexing. An embedding is a list of numbers that represents the meaning of a chunk or a question, so that pieces of text with similar meaning have similar numbers.

Dense retrieval compares these embeddings, while keyword retrieval compares the actual words, usually with the BM25 ranking function. A reranker is a slower and more careful model that rescores a short list of candidates returned by the first search.

On the verification side, natural language inference (NLI) is the task of deciding whether a premise passage entails, contradicts or is neutral towards a hypothesis sentence. In this system the cited passage is the premise and each claim of the answer is the hypothesis. An atomic fact is one small statement inside a claim, such as a number of days, an amount of money or the name of a party. The Verification Confidence Score (VCS) is the weighted score that combines the results of the checks for every claim, and the abstention threshold is the value of this score below which the system refuses to answer. Summary-augmented chunking (SAC) is the chunking method used for statutes, in which every chunk is stored together with a short summary of the section it belongs to. These terms are listed again with their full forms in the List of Abbreviations at the beginning of this report.

1.1.3 Why Legal Question Answering Is Difficult

Legal text has properties that make automatic question answering harder than in many other domains. Statutes are organised as a hierarchy of Acts, Chapters, Sections and Clauses, and the meaning of a clause often depends on the section that contains it. A clause that says "within thirty days" means nothing on its own unless the reader knows what the thirty days apply to. Sections also refer to each other, define terms in one place and use them in another, and contain exceptions that reverse the general rule. Contracts have a different structure but the same problem: a definition at the start of an agreement changes the meaning of every later clause that uses the defined term. A system that cuts documents into fixed-size pieces of text, which is the default in many retrieval pipelines, can easily separate a clause from the context that gives it its meaning.

The second difficulty is that legal users need precision rather than a general impression. If a user asks for the punishment for theft, an answer that mentions imprisonment but gives the wrong maximum term is worse than no answer, because it looks reliable. The user also needs to know which provision the answer relies on, so that it can be read and checked. Finally, the vocabulary of ordinary users is very different from the vocabulary of the law. A person may ask what happens if a landlord throws them out by force, while the statute speaks of unlawful eviction. A useful system must connect such paraphrases to the right provisions, keep exact legal terms and section numbers searchable, and still refuse to answer when the indexed documents simply do not cover the question that was asked.

1.2 Motivation

The motivation for this project comes from real failures of AI tools in legal work. In Mata v. Avianca, Inc., a federal court in the United States sanctioned lawyers who had filed a brief that cited court decisions invented by a chatbot [6]. The lawyers had not checked the citations, and the cases looked convincing because they had plausible names, dates and quotations. This case is now widely discussed because it shows how a fluent model can mislead even trained professionals. The same risk applies to students, small businesses and ordinary citizens who use chatbots to understand their rights, and who have even less ability to tell a real provision from an invented one. A tool that answers legal questions should therefore make checking easy, and should not present an answer as reliable unless the evidence behind it has been examined.

A second motivation is that retrieval alone has not solved the problem. An evaluation of commercial legal research tools, which are built on retrieval and marketed as reliable, found that they still produce hallucinated answers in a significant share of queries [7]. Retrieval reduces the chance that the model invents a source, but it does not guarantee that the answer the model finally writes matches the passages it was given. This observation led us to focus on the step that most systems leave out: checking the generated answer against its evidence before the user sees it. We also wanted the result of this check to be visible and understandable, so that a user can see not only that an answer was accepted or refused but also why, claim by claim, in a form that a teacher, a reviewer or a lawyer could audit.

1.3 Problem Statement

AI chatbots answer legal questions confidently, but they can state rules and citations that do not exist, and in a standard retrieval-augmented system nothing checks the final answer against the retrieved evidence before it reaches the user. Three weaknesses lie behind this problem. Fixed-size chunking separates a clause from the section that gives it meaning, so the retrieved text may be correct in isolation but misleading in context. Retrieval ranks passages by their similarity to the question, which is not the same as supporting the answer the model then writes. And the generated answer is shown without any verification, so a wrong claim with a plausible citation looks exactly like a correct one. Existing work addresses these issues separately, and few systems decide automatically whether an answer should be shown at all.

The problem addressed in this project is therefore to design and build a legal question-answering system that keeps the legal structure of documents during indexing, retrieves evidence with both meaning-based and keyword-based search, forces the language model to cite a source passage for every claim, verifies each claim against its cited

passage with independent checks, and combines these checks into a single score that decides whether the answer is shown or withheld. The system must also record a proof for every answer so that its decision can be inspected, must refuse to answer when the documents do not support an answer, and must be evaluated honestly, including the cases in which it refuses an answer that it should have given. The evaluation must show which part of the design is actually responsible for keeping wrong claims away from the user.

1.4 Research Objectives

The objectives of this project follow directly from the problem statement. They cover the way documents are prepared, the way evidence is retrieved, the way answers are written and checked, the way the decision to show or withhold an answer is made, and the way the whole system is evaluated and made available to users. Each objective was translated into one or more components of the implementation described in Chapters 4 and 5, and each one is measured in Chapter 6 wherever a measurement was possible. The objectives were also chosen so that the work could be completed within the time and computing resources available to an undergraduate team, which is why the system relies on small local models for retrieval and verification and uses an external service only for writing the answer. The six research objectives are stated below.

To construct an ingestion and chunking pipeline that reads statutes and contracts in PDF and plain text form, detects whether a document has an Act, Chapter, Section and Clause structure, and splits it along that structure using summary-augmented chunking, so that every chunk is one section or clause with a stable identifier and a short summary of the section it belongs to. For documents without such structure, such as contracts, the pipeline must fall back to paragraph chunking and record which strategy was used, so that the rest of the system and the evaluation can tell the two cases apart. The pipeline must also detect duplicate uploads with a content fingerprint, so that the same document is never indexed twice and the search results are not distorted by repeated text.

To develop a hybrid retrieval component that searches the indexed chunks with dense embeddings and with BM25 keyword search at the same time, merges the two rankings with reciprocal rank fusion, and reranks a deliberately large pool of candidates with a cross-encoder before the best passages are passed to the language model. The aim of this objective is to find the correct provision both when the user paraphrases the law in everyday words and when the user mentions an exact legal term or section number. The component must run on an ordinary computer without a graphics processor, must store its index in a single local database that needs no separate server, and must expose its

intermediate results so that retrieval can be inspected on its own before any answer is generated.

To design a citation-forced generation step in which the language model may use only the retrieved passages, must write one claim per line, and must end every claim with the identifier of the passage that supports it. When the passages cannot answer the question, the model must return a fixed abstention marker instead of an answer. This strict format is what makes automatic verification possible, because every claim can be matched to its evidence without guessing, and it also allows the language model to be replaced by another provider without changing the rest of the pipeline. A separate path must handle questions about general legal concepts that no indexed document defines, and must label such answers clearly as general knowledge so that they can never be mistaken for sourced answers.

To implement a six-layer verification chain that checks that every cited passage was actually retrieved, tests with a natural language inference model whether the passage entails the claim, checks the small facts inside each claim, optionally measures whether the claim recurs when the answer is generated again, combines these results into a Verification Confidence Score, and writes a proof object that records every verdict. The score must decide whether the answer is shown, with a threshold of 0.60, and the proof must be visible in the user interface when a citation is clicked. Each layer must be able to be switched off individually, so that its contribution to the final decision can be measured by removing it and observing how the behaviour of the system changes.

To evaluate the system honestly through ablation experiments and an end-to-end test. The ablations must compare section-aware chunking with paragraph chunking under several retrieval settings, and must remove one verification layer at a time while deliberately corrupted answers are passed through the chain. The end-to-end test must use the real web interface, real documents and questions whose expected answers were written and recorded before the test was run, so that the results could not be adjusted afterwards. Every number reported must come from logged result files kept in the project repository, and the evaluation must report false refusals and other failures as clearly as successes, since a system that refuses too often is also a problem for its users.

To build a usable web application on top of the pipeline, with pages for uploading documents, asking questions, inspecting retrieval and viewing evaluation results, and to make the complete project reproducible and available. This includes support for uploading many documents at once, a ready-made pack of Indian criminal law documents for demonstrations, documentation of the design and the experiments, and configuration that allows the application to be hosted online at no cost. All code, data

preparation scripts, experiment logs and documentation must be kept in a public GitHub repository, so that the work can be reviewed by the department and reused or extended by other students working on similar problems in the future.

1.5 Scope and Limitations

The scope of this project is a working prototype of a verified legal question-answering system and a careful measurement of how its parts behave. The system handles statutes with a recognisable Act, Chapter, Section and Clause hierarchy, such as the five synthetic statutes we wrote for the experiments and the twenty-seven files of Indian criminal law prepared for demonstrations, as well as contracts without such structure, such as the non-disclosure agreements of the ContractNLI dataset. It answers questions in English, from documents uploaded by the user, and it shows a proof for every verified answer. The evaluation covers retrieval quality, the contribution of each verification layer, the behaviour of the full system through its user interface, and the cost of building the pipeline with LangChain compared with plain Python code.

1.5.1 Assumptions

The project makes several assumptions. It assumes that the documents uploaded by the user are the authoritative source for the questions asked, so that an answer supported by these documents is considered correct even if the law has since changed. This matters for the Indian criminal law pack, which contains the Indian Penal Code, the Code of Criminal Procedure and the Indian Evidence Act as they stood before 1 July 2024, when they were replaced by the Bharatiya Nyaya Sanhita, the Bharatiya Nagarik Suraksha Sanhita and the Bharatiya Sakshya Adhiniyam. It assumes that text can be extracted from the uploaded files, which excludes scanned images without a text layer. It also assumes access to a language model service for writing answers, and that the user reads the proof when a decision matters rather than treating the badge alone as final legal advice.

1.5.2 Outside the Scope

Several related problems are outside the scope of this project. The system does not give legal advice and is not intended to replace a lawyer; it explains what the uploaded documents say and shows where they say it. It does not track amendments or check whether an uploaded statute is still in force. It does not handle languages other than English, handwritten or scanned documents, or reasoning that combines many provisions into a new legal argument. The verification chain checks that each claim is supported by the passage it cites, but it cannot detect that an answer has left out something important, a limitation discussed in detail in Chapter 6. Finally, the threshold of the confidence score has not been calibrated on a large held-out set, and the

evaluation uses small question sets that were sufficient to test the design but not to rank retrieval methods against each other.

1.6 Organization of the Report

This report is organised into seven chapters. Chapter 1 has introduced the background of the project, the key terms used throughout the report, the motivation, the problem statement, the research objectives and the scope. Chapter 2 reviews related work on hallucination in language models, retrieval-augmented generation, chunking strategies for legal text, hybrid retrieval and reranking, verification of generated answers, and legal datasets and benchmarks, and it identifies the research gap addressed by this project. Chapter 3 analyses existing approaches and their drawbacks, describes the proposed system and its feasibility, and lists the hardware and software requirements together with the functional and non-functional requirements of the system. Chapter 4 presents the system design and methodology, including the overall pipeline, the architecture, the modules, the formulas used for fusion and scoring, the data design and the UML diagrams.

Chapter 5 describes the implementation, including the experimental setup, the structure of the project, the datasets used, the key parameters, selected code from the repository and the deployment of the web interface. Chapter 6 presents the results and discussion, covering the evaluation metrics, the testing strategy, the functional tests, the quantitative results of the retrieval ablation, the verification ablation, the end-to-end test and the comparison between the LangChain and plain Python versions, followed by an error analysis, sample outputs and a discussion of ethical and responsible use. Chapter 7 concludes the report, states the limitations of the current system and proposes future enhancements. The report ends with the list of references cited in the text and appendices that describe the source code repository, how to run and host the system, and the demonstration questions used with the Indian criminal law documents.

CHAPTER 2
LITERATURE SURVEY

2.1 Review of Related Work

The literature relevant to this project comes from several areas that have developed largely in parallel. Work on language models explains why hallucination happens and how often it appears in legal tasks. Work on retrieval-augmented generation describes the standard way of grounding answers in documents. Research on chunking, embedding, keyword ranking, rank fusion and reranking determines how well the right evidence is found. Research on natural language inference, atomic fact checking and self-consistency provides ways to check a generated answer against evidence. Finally, legal datasets and benchmarks provide the material on which such systems can be tested. This chapter reviews each area in turn, summarises the reviewed work in a table, and explains how the present project combines these ideas and which gap it addresses that the reviewed work leaves open.

2.1.1 Large Language Models and Hallucination

Modern language models are based on the transformer architecture introduced by Vaswani et al. [1], which replaced recurrence with attention and made it practical to train very large models on very large corpora. Pretrained encoders such as BERT [2] showed that a model trained on general text can be fine-tuned for many specific tasks, and later generative models extended this idea to open-ended text generation. A detailed survey by Ji et al. [3] defines hallucination as generated content that is nonsensical or unfaithful to the provided source, and distinguishes intrinsic hallucination, which contradicts the source, from extrinsic hallucination, which cannot be verified from the source at all. The survey also reviews detection and mitigation methods and concludes that no single method removes the problem, which is one reason why this project combines several independent checks rather than relying on one detector.

In the legal domain, Dahl et al. [4] profiled the legal hallucinations of several widely used language models on questions about real court cases and found that the models frequently gave incorrect answers with confidence, especially for less prominent cases and lower courts. Magesh et al. [7] then evaluated commercial legal research tools that use retrieval and are marketed as reducing hallucination, and found that they still produced incorrect or misgrounded answers for a substantial share of queries. The practical consequences are illustrated by Mata v. Avianca, Inc. [6], in which lawyers were sanctioned for citing cases that a chatbot had invented. Together these works establish that hallucination in legal question answering is frequent, that retrieval

reduces but does not remove it, and that unchecked answers can cause real harm to the people who rely on them.

2.1.2 Retrieval-Augmented Generation

Retrieval-augmented generation was introduced by Lewis et al. [5], who combined a neural retriever with a sequence-to-sequence generator so that the model could draw on an external document collection when producing an answer. This design made it possible to update a model's knowledge by changing the documents rather than retraining the model, and it improved factual accuracy on knowledge-intensive tasks. Dense passage retrieval, described by Karpukhin et al. [8], showed that passages and questions encoded as dense vectors by trained encoders can outperform traditional keyword retrieval for open-domain question answering. A later survey by Gao et al. [9] organises the large body of work on retrieval-augmented generation into naive, advanced and modular designs, and discusses improvements to indexing, retrieval, reranking and generation, as well as the evaluation of such systems.

The survey by Gao et al. [9] also points out a weakness that is central to this project: most retrieval-augmented systems treat the retrieved passages as context for the generator but do not check whether the generated answer is actually supported by them. Retrieval guarantees that relevant text was available, not that the model used it correctly. The model can mix information from different passages, attach a claim to the wrong source, or fall back on its own memory when the passages are incomplete. Our system follows the general retrieval-augmented design of Lewis et al. [5] and uses dense retrieval in the spirit of Karpukhin et al. [8], but it adds a verification stage after generation and a decision rule that can withhold the answer, which turns the retrieved passages from mere context into evidence that each claim must be shown to rest on.

2.1.3 Chunking Strategies for Legal Documents

Before retrieval, documents must be split into chunks, and the way they are split strongly affects what can be found. Reuter et al. [10] studied retrieval for large legal datasets and observed that standard chunking loses the context that tells a reader which document and which part of it a passage comes from. They proposed summary-augmented chunking, in which each chunk is enriched with a summary of the whole document written by a language model, and showed that this reduces retrieval errors in which a passage from the wrong document is returned. Our project adopts the same idea at a finer level and without a model call: for statutes, each chunk carries an extractive summary of its own section, made of the section heading and its first sentence, so that ingestion stays fast and deterministic and every chunk still knows which provision it belongs to.

Domain adaptation of language representations is a related line of work. Chalkidis et al. [11] trained LEGAL-BERT on legal text and showed that models adapted to legal language perform better than general models on several legal classification and extraction tasks. This finding is relevant to our system in two ways. First, it suggests that the general-purpose models we use for embeddings and natural language inference may miss distinctions that legal experts consider important, which is consistent with the false refusals we observed and report in Chapter 6. Second, it shows a clear path for future improvement, since the embedding model or the inference model in our pipeline could be replaced by a legally adapted model without changing the overall design. We chose general models for this project because they are small, freely available and well documented, which suited the resources of the project.

2.1.4 Hybrid Retrieval and Reranking

Keyword retrieval remains a strong baseline. The BM25 ranking function, described in detail by Robertson and Zaragoza [12], scores a document by how often the query terms occur in it, adjusted for document length and for how rare each term is across the collection. It is especially effective for exact terms such as section numbers, defined words and names, which dense models can blur. Sentence-BERT, introduced by Reimers and Gurevych [13], made dense sentence embeddings practical by training siamese networks that produce vectors which can be compared with cosine similarity, and the all-MiniLM-L6-v2 model used in this project belongs to this family. Because the two kinds of retrieval fail in different ways, combining them is a common way to improve recall, and it is the approach taken in our system for both statutes and contracts.

Combining two ranked lists requires a fusion method. Cormack et al. [14] proposed reciprocal rank fusion, which gives each document a score equal to the sum of one divided by a constant plus its rank in each list, and showed that this simple rule outperforms more complex learned methods. Its main advantage is that it uses only ranks, so the very different score scales of BM25 and cosine similarity do not need to be calibrated. After fusion, a reranker can examine the best candidates more carefully. Nogueira and Cho [15] showed that a BERT model that reads the query and a passage together, known as a cross-encoder, reranks passages much more accurately than first-stage retrieval. Our system uses bge-reranker-base, a cross-encoder released with the C-Pack resources of Xiao et al. [16], to rerank a pool of one hundred fused candidates before the best twelve are passed to the language model.

2.1.5 Verification of Generated Answers

Natural language inference provides a direct way to test whether a passage supports a claim. The SNLI corpus of Bowman et al. [17] and the broader MultiNLI corpus of Williams et al. [18] made it possible to train models that classify a premise and

hypothesis pair as entailment, contradiction or neutral. RoBERTa, described by Liu et al. [19], improved the pretraining of BERT-style models, and the roberta-large-mnli model, which is RoBERTa fine-tuned on MultiNLI, is a widely used and freely available inference model. Our verification layer V2 uses this model with the cited passage as the premise and the claim as the hypothesis. A CONTRADICTS verdict sets the score of the claim to zero, while an ENTAILS verdict contributes the largest weight to the claim score, so that the inference check is the main signal about whether a claim is supported.

Two further ideas inform the other verification layers. Min et al. [20] introduced FActScore, which breaks a long generated text into atomic facts and measures the fraction of them supported by a knowledge source; our layer V3 applies the same idea to each claim and its cited passage. Manakul et al. [21] proposed SelfCheckGPT, which detects hallucination by sampling several answers to the same prompt and checking whether a statement recurs, on the reasoning that invented details vary between samples while grounded facts remain stable; our optional layer V4 follows this approach. In the legal domain, Figueiredo et al. [22] built a multi-stage anti-hallucination pipeline for Brazilian legal texts that audits citations after generation, and Maghakian et al. [23] proposed an embedding-free design that moves retrieval into the language model at a higher cost per query. These systems check or improve parts of the process, but they do not reduce the checks to a single score that decides whether the answer is shown.

2.1.6 Legal Datasets and Benchmarks

Progress in legal question answering depends on datasets in which the correct answers are known. LegalBench, assembled by Guha et al. [24], collects a large set of legal reasoning tasks contributed by legal professionals and is used to measure the reasoning abilities of language models. LegalBench-RAG, introduced by Pipitone and Alami [25], focuses specifically on the retrieval step and measures whether a system retrieves the precise text spans that answer a legal question. RAGTruth, created by Niu et al. [26], is a corpus of answers produced by retrieval-augmented systems in which hallucinated spans have been annotated by people, and it is used to develop and test hallucination detectors. These benchmarks measure the problem carefully, but they do not themselves act on it, which is the step our system adds by checking each answer and deciding whether to show it.

Several datasets of real contracts were used in this project for testing on documents that the team did not write. ContractNLI, created by Koreeda and Manning [27], contains non-disclosure agreements annotated for document-level natural language inference, and its agreements were used in our batch test and end-to-end test. CUAD, released by Hendrycks et al. [28], contains commercial contracts annotated by legal experts for

clauses that matter in contract review, and a non-competition agreement from it was used in the end-to-end test. MAUD, released by Wang et al. [29], contains merger agreements annotated for deal points. For demonstrations with Indian law, we prepared a pack of criminal law documents from a public dataset of Indian Acts [30], whose text corresponds to the Acts published by the Government of India on India Code [31], as described in Chapter 5.

2.1.7 Positioning of the Present Work

The present work brings these separate lines of research into one system and adds a decision rule on top of them. It adopts summary-augmented chunking from Reuter et al. [10] in a section-level, extractive form; dense embeddings from the Sentence-BERT family [13]; BM25 keyword retrieval [12]; reciprocal rank fusion [14]; and cross-encoder reranking [15], all inside a single local LanceDB index [32]. On the generation side it forces a citation for every claim, and on the verification side it combines an inference check, an atomic fact check and an optional self-consistency check into one Verification Confidence Score that gates the answer. The contribution of the project is not any single component but the way the components are joined, the gate that acts on their output, the proof that records the decision, and an evaluation that measures which part of the design actually keeps wrong claims away from the user.

2.2 Summary of Literature

Table 2.1 summarises the main works reviewed in this chapter, the problem each addresses, the method it proposes and the way its ideas are used or extended in this project. The table shows that the reviewed work can be grouped into three kinds. The first kind measures the problem of hallucination and shows that it is frequent and harmful in legal settings. The second kind improves one stage of the retrieval-augmented pipeline, such as chunking, retrieval, fusion or reranking. The third kind checks generated text after it is written, either by inference, by atomic facts, by self-consistency or by auditing citations. None of the reviewed works combines improvements in all three stages with a single score that decides whether an answer is shown, together with a proof that a user can inspect, which is the combination implemented and evaluated in this project.

Table 2.1 Summary of Reviewed Literature

Work	Focus	Method	Use in this project
Ji et al. [3]	Hallucination survey	Taxonomy of hallucination and mitigation methods	Motivates combining several checks
Dahl et al. [4]	Legal hallucination	Profiles model errors on real cases	Shows the size of the problem

Work	Focus	Method	Use in this project
Magesh et al. [7]	Legal research tools	Evaluates retrieval-based products	Shows retrieval alone is not enough
Lewis et al. [5]	RAG	Retriever plus generator	Base design of the system
Reuter et al. [10]	Legal chunking	Summary-augmented chunking	Adapted as section-level SAC
Robertson and Zaragoza [12]	Keyword ranking	BM25 probabilistic model	Keyword half of hybrid search
Reimers and Gurevych [13]	Sentence embeddings	Siamese BERT networks	MiniLM embedding model
Cormack et al. [14]	Rank fusion	Reciprocal rank fusion	Merges dense and BM25 lists
Nogueira and Cho [15]	Reranking	BERT cross-encoder	bge-reranker-base stage
Williams et al. [18]	NLI data	MultiNLI corpus	Training data of the V2 model
Min et al. [20]	Factual precision	Atomic fact scoring	V3 fidelity layer
Manakul et al. [21]	Self-consistency	Sampling-based detection	V4 consistency layer
Figueiredo et al. [22]	Legal RAG checks	Post-generation citation audit	Compared; we add a gate
Pipitone and Alami [25]	Legal RAG benchmark	Span-level retrieval test	Future evaluation target

2.3 Research Gap

The review shows a clear gap. Work on legal hallucination measures how often models and tools are wrong, but it does not propose a mechanism that stops wrong answers from being shown. Work on chunking and retrieval improves the evidence given to the model, but it does not check what the model does with that evidence. Work on verification provides useful signals, such as entailment verdicts, atomic fact scores and self-consistency, but these signals are usually reported as scores for analysis rather than used as a decision rule inside a user-facing system. Post-generation citation audits in legal systems come closest to our aim, yet they still leave open the question of how the separate checks should be combined and what the system should do when they disagree, and they rarely show the evidence for the decision in a form an ordinary user can read.

This project addresses that gap in three ways. First, it joins structure-preserving chunking, hybrid retrieval with reranking and citation-forced generation into one pipeline, so that every claim of an answer can be traced to a specific section or clause. Second, it reduces the citation, entailment, fidelity and consistency checks to a single

Verification Confidence Score and uses that score as a gate that either shows the answer or withholds it, with a proof object recording every verdict. Third, it evaluates the design by removing one layer at a time and by testing the whole system through its user interface with answers fixed in advance, which shows that the gate, rather than any single detector, is what keeps known-bad claims away from the user. The evaluation also measures the cost of this caution in false refusals, which is the main open problem identified by this work.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing Approaches and their Drawbacks

The simplest existing approach is to ask a general-purpose chatbot directly. This is fast and convenient, and the answers are often well written, but the model answers from patterns learned during training rather than from a specific document. It may not know the current version of a statute, it may confuse provisions from different laws, and it may invent sections or cases that do not exist, as the studies discussed in Chapter 2 have shown [4]. Most importantly, the user has no reliable way to check the answer, because the model either gives no source or gives a source that may itself be invented. For legal questions, where a wrong answer can cause real harm, this approach is not acceptable as a basis for decisions, even though it remains useful for general orientation when the user already knows the topic well.

The standard improvement is a retrieval-augmented system in which documents are split into fixed-size chunks, embedded and stored in a vector database, and the most similar chunks are given to the model as context [5]. This grounds the answer in real text and allows the system to cite the chunks it used. However, fixed-size chunking cuts sections in arbitrary places and loses their context, similarity search can miss exact legal terms, and the answer the model writes is not checked against the chunks it was given. Commercial legal research tools built on retrieval reduce hallucination but, as Magesh et al. [7] found, still produce misgrounded answers. These systems usually show their sources, but they do not tell the user whether each claim is actually supported, and they do not refuse to answer when the support is weak.

A third group of approaches adds checks after generation. Some systems audit the citations in an answer [22], others score the answer with a hallucination detector trained on annotated data [26], and others compare several generated answers for consistency [21]. These checks are valuable, but in most systems they are reported to the developer or shown as a warning rather than used to decide whether the answer should reach the user at all. They also tend to operate on the answer as a whole, which hides the fact that one claim may be well supported while another next to it is not. The drawbacks of the existing approaches therefore come down to three points: the context of legal provisions is lost during chunking, the final answer is not systematically verified, and there is no clear, inspectable rule for refusing an answer.

3.2 Proposed System

The proposed system, Legal AI, addresses these drawbacks with a pipeline in which every stage is designed with verification in mind. Documents are split by their legal structure where it exists, so each chunk is a complete section or clause with an identifier such as ipc_theft::s379 and a short summary of its section. Retrieval combines meaning-based and keyword-based search, merges them with reciprocal rank fusion and reranks a large candidate pool with a cross-encoder. The language model must write one claim per line, each ending with the identifier of its supporting chunk, or return an abstention marker. Each claim is then checked by six verification layers, which produce a Verification Confidence Score, and the answer is shown only when this score is at least 0.60. Every answer carries a proof object that the user can open from its citations.

The proposed system also includes the practical features needed to use and evaluate it. A web application provides pages for uploading many documents at once, asking questions in a chat interface, inspecting retrieval results from dense, keyword and hybrid search side by side, and viewing the logged experiments and a map of the indexed chunks. Duplicate uploads are detected and can be replaced. Questions about general legal concepts are answered from the model's own knowledge but labelled clearly as general knowledge, without citations or a score. The back end exposes every step as an API call, so the same pipeline can be driven by scripts for batch testing. Finally, the project includes a demonstration pack of Indian criminal law, scripts to load and test it, and configuration for hosting the complete application online on a free cloud development environment.

3.3 Feasibility Study

Before implementation, the feasibility of the project was considered from technical, economic, operational and legal points of view, and the conclusions are summarised in Table 3.1. Technically, every component required by the design is available as mature open-source software, and the three local models are small enough to run on a computer without a graphics processor. Economically, the project needs no paid software, since the language model service used for generation offers a free tier and the documents used are either public datasets or texts we wrote ourselves. Operationally, the system is used through a web browser and needs no special training beyond understanding the badge and the proof view. From a legal and ethical point of view, the system works only on documents supplied by the user, and it is presented as a tool for reading documents, not as a source of legal advice.

Table 3.1 Feasibility Study Summary

Aspect	Assessment	Conclusion
Technical	Open-source libraries for every stage; models run on a CPU with about 4 GB of memory	Feasible
Economic	No licence costs; free tier of the language model service; free hosting option	Feasible
Operational	Browser-based interface; clear Verified and Abstained badges; proof on click	Feasible
Schedule	Built in phases, each with its own tests and documentation	Feasible
Legal and ethical	Uses public or self-written documents; not presented as legal advice	Feasible with disclaimer

3.4 Hardware Requirements

The hardware requirements are modest because the system was designed to run on ordinary computers. The heaviest components are the three local models: the embedding model needs about 90 megabytes of memory, the reranker about 1.1 gigabytes and the natural language inference model about 1.4 gigabytes, and together with the Python runtime and the libraries the back end needs roughly four gigabytes of memory while it is running. No graphics processor is required, although one would make reranking and verification faster. Disk space is needed for the installed libraries, the downloaded model weights and the index, which together occupy about five gigabytes. An internet connection is required to download the models once and to call the language model service that writes the answers. Table 3.2 lists the minimum and recommended configuration for running the system.

Table 3.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 64-bit CPU	Quad-core 64-bit CPU
Memory (RAM)	8 GB	16 GB
Storage	10 GB free disk space	20 GB SSD
Graphics processor	Not required	Optional, speeds up reranking and NLI
Network	Internet connection for model download and LLM API	Stable broadband connection

3.5 Software Requirements

The software of the project is divided into the back end, written in Python, and the front end, written in JavaScript. The back end uses LangChain [33] to compose loading,

retrieval and generation, LanceDB [32] as the embedded vector database, sentence-transformers and Hugging Face Transformers to run the local models, and FastAPI [34] with the Uvicorn server to expose the pipeline as a web API. The front end is a single-page application built with React [35], the Vite build tool and the Tailwind CSS styling framework. The language model is accessed through the Groq API, and adapters also exist for other providers. Development used Git and GitHub for version control and pytest for automated testing. Table 3.3 lists the main software components, and the subsections that follow state the functional requirements, the non-functional requirements and the development environment.

Table 3.3 Software Requirements

Category	Software	Purpose
Language (back end)	Python 3.11	Pipeline, API and experiments
Framework	LangChain	Composition of retrieval and generation
Vector database	LanceDB	Chunks, vectors and metadata on disk
Models	sentence-transformers, Transformers, PyTorch (CPU)	Embeddings, reranking and NLI
Keyword search	rank_bm25	BM25 ranking
Web API	FastAPI, Uvicorn	REST endpoints for the web app
Language (front end)	JavaScript, Node.js 20	Web application
Front end	React 18, Vite, Tailwind CSS	User interface
LLM service	Groq API (gpt-oss-120b)	Writing the answers
Tools	Git, GitHub, pytest, Playwright	Version control and testing

3.5.1 Functional Requirements

The functional requirements describe what the system must do. The system must accept one or many documents in PDF or plain text form, extract their text, detect duplicates by content fingerprint and by name, and either reject a duplicate or replace the indexed copy at the user's request. It must split statutes by section and clause with summary-augmented chunking, fall back to paragraph chunking for other documents, and report which strategy was used and how many chunks were created. It must embed every chunk, store it in the index together with its metadata, and allow a document to be deleted. For every question it must retrieve candidates with dense and keyword search, fuse and rerank them, and generate an answer in which every claim cites a chunk, or return an abstention when the context is insufficient.

The system must then verify every claim with the six-layer chain, compute the Verification Confidence Score, and show the answer only when the score reaches the threshold; otherwise it must show an abstention that explains that the documents do not support an answer. It must produce a proof object for every answer and display it when the user clicks a citation, including the cited passage and the verdict of each layer. It must answer general concept questions from model knowledge only when they do not refer to the indexed documents, and must label such answers as general knowledge. It must keep a history of conversations, offer a search page that shows the results of dense, keyword and hybrid retrieval side by side, and offer an evaluation page that lists logged experiments and shows a two-dimensional map of the indexed chunks.

3.5.2 Non Functional Requirements

The non-functional requirements describe the qualities the system must have. Reliability is the most important: the system must never present an unsupported claim as verified, and it must prefer refusing an answer to showing one it cannot support. Transparency is closely related, since every decision must be explainable through the proof object, which lists the evidence and the verdicts behind it. Reproducibility requires that every experiment be run from a stored configuration and that every reported number be read from logged result files in the repository. Usability requires a clean interface with clear badges, readable citations and helpful error messages, for example when the back end is not running or a document is already indexed. Performance requirements are modest: a question should be answered in under half a minute on an ordinary computer, and uploading a statute should take seconds once the models are loaded.

Maintainability and portability are also required. The pipeline is divided into modules for ingestion, chunking, embedding, indexing, retrieval, generation and verification, each with its own tests, so that one part can be changed without breaking the others. The language model sits behind a small adapter, so the provider can be changed with a configuration setting. The index is a folder on disk with no database server, so the system can be installed with the standard Python and Node.js package managers on Windows, Linux or macOS, and it can also be packaged as a Docker container or started in a cloud development environment. Security requirements are limited but important: API keys are kept in environment variables or hosting secrets and never committed to the repository, uploaded documents stay on the machine running the system, and the only data sent outside is the prompt containing the retrieved passages.

3.5.3 Development Environment

The system was developed on personal laptops running Windows and tested in Linux environments. The back end runs in a Python 3.11 virtual environment created from the

project's requirement files, and the front end runs on Node.js 20 with the Vite development server, which reloads the interface instantly when the code changes. A single batch file starts both servers and opens the browser, which made it easy for every team member to run the complete system. Visual Studio Code was used as the main editor, GitHub was used to share the code and track every change, and pytest was used to run the unit and API tests of each module. The pipeline is composed with LangChain [33] on top of LanceDB [32], served with FastAPI [34], and presented through a React [35] interface. Playwright was used to drive the web interface automatically during the end-to-end test, so that every upload, question and click was a real browser action.

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

4.1 Overall Pipeline

The design of Legal AI follows the path of a document and a question through the system, and Fig. 4.1 shows the complete flow. A document is first uploaded as a PDF or plain text file. Its text is extracted, cleaned and fingerprinted, and a duplicate of a document already in the index is rejected. The text is then examined for legal structure: if an Act, Chapter, Section and Clause hierarchy is found, the document is split with summary-augmented chunking; otherwise it is split into paragraphs. Every chunk is embedded and written to the LanceDB index. When a question arrives, the system runs dense and keyword retrieval, merges the results with reciprocal rank fusion, reranks the candidates with a cross-encoder and passes the best ones to the language model, which writes a cited answer.

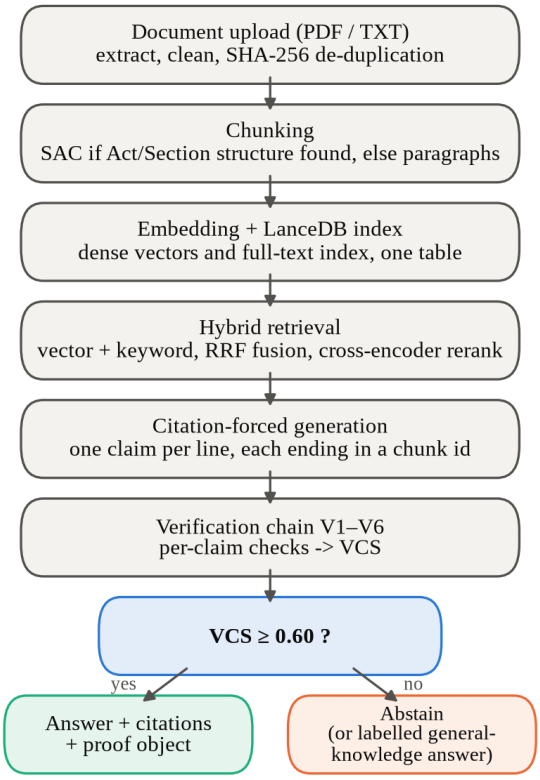


Fig. 4.1 Processing pipeline of Legal AI, from document upload to the decision to answer or abstain

The final stages of the pipeline are what distinguish this system from a standard retrieval-augmented design. The generated answer is split into claims, and every claim passes through the verification chain, which checks its citation, tests whether the cited passage entails it, checks its atomic facts and, when enabled, measures whether it recurs

across samples. The scores of all the claims are combined into the Verification Confidence Score. If the score is at least 0.60, the answer is shown with its citations and a proof object; otherwise the system abstains. The figure also shows the one exception to this rule: a question about a general legal concept that is not defined in the indexed documents is answered from the model's knowledge and labelled clearly as an unverified general-knowledge answer, without citations or a score, so that it can never be confused with a sourced answer.

4.2 System Architecture

The system is organised in three tiers, shown in Fig. 4.2. The presentation tier is a React single-page application that runs in the user's browser and provides the Home, Ingest, Ask, Search and Evaluation pages. It never touches the pipeline directly; every action is sent as an HTTP request to the application tier and every result comes back as JSON. The application tier is a FastAPI service that exposes endpoints such as /ingest, /query, /retrieve, /stats and /evaluation, and it contains the modules for ingestion and chunking, hybrid retrieval and reranking, citation-forced generation and verification. The data and model tier holds the LanceDB index, which stores chunks, vectors and metadata in a folder on disk, the three local models used for embeddings, reranking and natural language inference, and the external Groq service that runs the language model used to write answers.

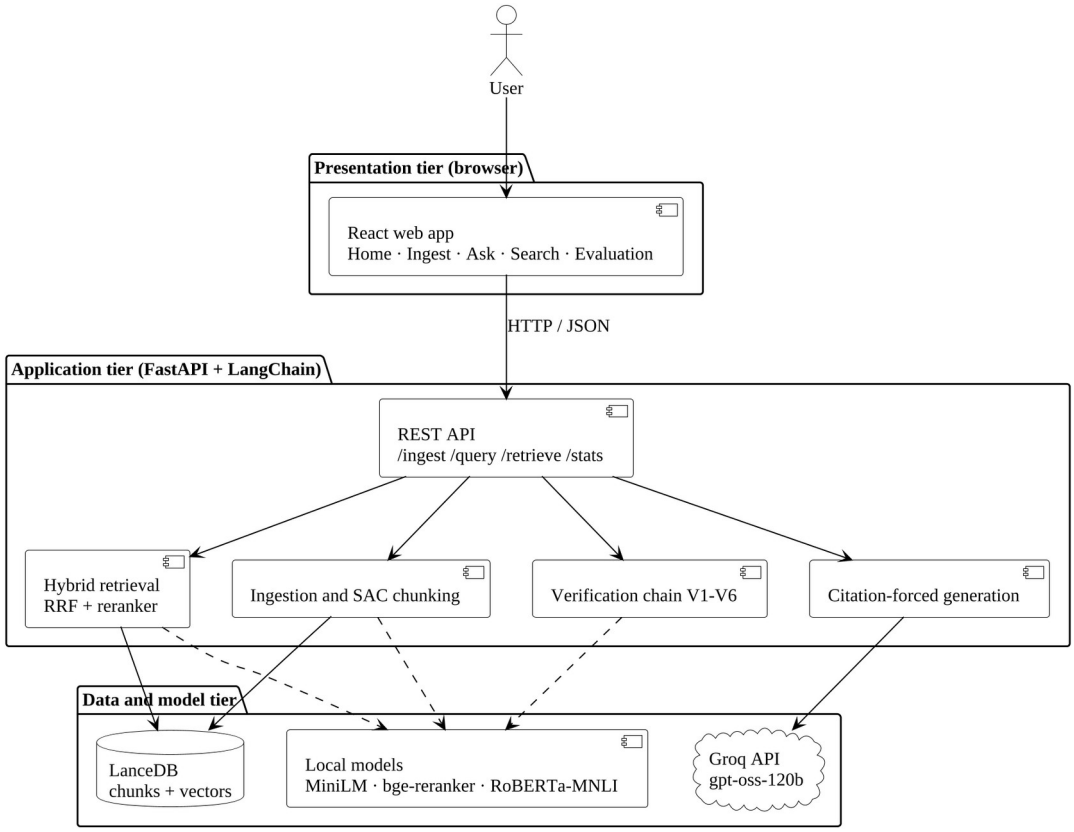


Fig. 4.2 Three-tier system architecture of Legal AI

This separation has several practical benefits. Because the web interface only talks to the API, the same pipeline can be driven by scripts that upload documents and ask questions in bulk, which is how the batch tests and the demonstration checks were run. Because the index is an embedded database rather than a server, the whole system can be started on a laptop with two commands, packaged as a single Docker container, or started in a cloud development environment. Because the language model is called through an adapter, the provider can be changed between Groq, Gemini, OpenAI and Anthropic models with a configuration setting. And because the verification layer is independent of the model that writes the answer, the decision to show or withhold an answer does not depend on the same model grading its own output, which is an important property for a system meant to catch that model's mistakes.

4.3 Module Description

The application tier is divided into modules that correspond to the stages of the pipeline. Each module has a single responsibility, its own data types and its own automated tests, and the modules communicate through simple records such as chunks, retrieved candidates, claims and verification results. This structure allowed the team to develop and test the stages independently, to replace one implementation of a stage with another, and to switch off individual verification layers for the ablation experiments. It also made it possible to build the pipeline twice, once in plain Python and once with LangChain, and to compare the two versions stage by stage. The following subsections describe each module, the input it receives, the processing it performs and the output it passes to the next stage of the pipeline.

4.3.1 Ingestion and Duplicate Detection Module

The ingestion module receives an uploaded file and turns it into a document record with its text and metadata. Plain text files are read directly, and PDF files are read page by page with the pypdf library. The text is kept exactly as extracted rather than stripped, because the structure parser depends on line breaks to recognise headings. Each upload is fingerprinted with the SHA-256 hash of its content, and its file name is turned into a source identifier. Before any chunking or embedding is done, the module checks the index for a document with the same content hash or the same source identifier. If one exists, the upload is rejected with a message that names the existing document and its number of chunks, and the user can choose to replace it, which deletes the indexed copy and indexes the new upload in its place. This prevents the same law from being stored twice, which would distort search results.

4.3.2 Summary-Augmented Chunking Module

The chunking module first runs a structure parser that looks for chapter headings, section headings such as "Section 379. Punishment for theft.", numbered sub-sections and lettered clauses. Section numbers may carry letter suffixes such as 498A or 13B, which the parser handles explicitly. When structure is found, every section produces a body chunk for its main text and one chunk for each of its clauses. Each chunk receives an identifier built from the source identifier, the section number and the clause label, such as ipc_theft::s379 or urban_tenancy_act_2019::s7:a, and a contextual summary made of the section heading and the first sentence of the section. The chunk also carries metadata naming the Act, the chapter, the section and the clause. When no structure is found, the module splits the text into paragraphs of at most 700 characters and records that the paragraph fallback was used.

4.3.3 Embedding and Indexing Module

The embedding module converts each chunk into a vector of 384 numbers with the all-MiniLM-L6-v2 sentence embedding model. For summary-augmented chunks, the input to the model is the contextual summary followed by the chunk text, so the vector encodes both the clause and the section that governs it, while the stored text remains the clause itself. The vectors are normalised so that cosine similarity can be computed with a dot product. The indexing module writes one row per chunk to a LanceDB table, holding the vector, the chunk identifier, the text, the summary, the content fingerprint and the metadata. LanceDB stores the table as files in a single folder, supports vector search directly and keeps versions of the table, so no separate database server is needed and the index can be deleted and rebuilt simply by removing the folder.

4.3.4 Hybrid Retrieval and Reranking Module

The retrieval module answers a question in two stages. In the first stage, the question is embedded with the same model as the chunks and compared with every chunk vector, and at the same time it is tokenised and scored against every chunk with BM25 keyword ranking. Each search returns its own ranked list, and the two lists are merged with reciprocal rank fusion using equal weights. In the Ask page, the fused list is cut to a candidate pool of one hundred chunks. In the second stage, the bge-reranker-base cross-encoder reads the question together with the summary and text of each candidate and gives it a relevance score, and the best twelve candidates are passed to the generator. The pool is much larger than the final number on purpose, because a reranker can only promote passages that the first stage actually returned, so a shallow pool limits quality however strong the reranker is.

4.3.5 Citation-Forced Generation Module

The generation module builds a prompt from the question and the twelve retrieved chunks and sends it to the language model. Each chunk is presented with its identifier in square brackets, its section reference and Act, its contextual summary and its text. The system prompt tells the model to use only these passages, to write each factual claim on its own line, to end every claim with the identifier of the passage that supports it, and never to write a factual sentence without a citation. If the passages do not contain enough information to answer, the model must reply with a single line that begins with a fixed abstention marker. A parser then reads the reply, recognises the abstention marker, splits the answer into claims and extracts the cited identifiers, tolerating common variations in bracket style. This fixed format is what allows every claim to be matched with its evidence by the verification module.

4.3.6 Verification Module

The verification module checks every claim with the six layers shown in Fig. 4.3. Layer V1 confirms that every chunk cited by the claim was among the retrieved chunks; a citation to an identifier that was never retrieved is treated as fabricated and sets the claim score to zero. Layer V2 runs the roberta-large-mnli inference model with the cited passage as premise and the claim as hypothesis; a CONTRADICTS verdict also sets the score to zero, while an ENTAILS verdict adds the entailment component. Layer V3 splits the claim into atomic facts and measures the fraction entailed by the passage. Layer V4, which is optional because it needs extra generations, measures how often the claim recurs when the answer is generated again. Layer V5 combines the components into claim scores and the answer-level Verification Confidence Score and applies the threshold, and layer V6 writes the proof object that records every verdict.

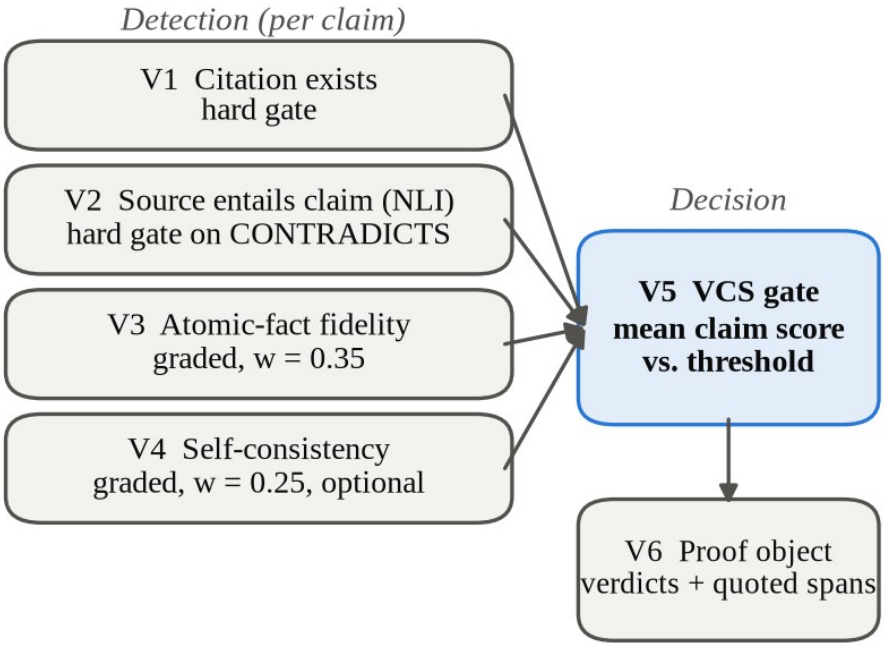


Fig. 4.3 The verification chain: V1 to V4 score each claim, V5 turns the scores into one decision and V6 records the evidence

4.3.7 Web Interface Module

The web interface is a React application with five pages. Home shows the size of the index and links to the other pages. Ingest accepts one or many files by drag and drop or through a file picker, uploads them one at a time, and shows for each file whether it was indexed, already indexed or rejected, how many chunks it produced and whether it was chunked by section or by paragraph, with a button to replace a duplicate and a view of the chunks. Ask provides a chat interface with saved conversations, in which every answer carries its badge, its score and clickable citations that open the proof view. Search runs one query through dense, keyword and hybrid retrieval side by side so that retrieval can be inspected before any generation. Evaluation lists the logged experiments with their headline numbers and shows a two-dimensional projection of every indexed chunk.

4.4 Model Selection and Justification

Four models are used in the pipeline, and Table 4.1 lists them with the reasons for their selection. The embedding model, all-MiniLM-L6-v2 from the Sentence-BERT family [13], was chosen because it is small, fast on a CPU and widely used for semantic search, and its limit of about 256 input tokens fits the short chunks produced by summary-augmented chunking. The reranker, bge-reranker-base [16], is a cross-encoder of moderate size that runs on a CPU and gives clearly better ordering than first-stage retrieval. The inference model, roberta-large-mnli, is RoBERTa [19] fine-tuned on

MultiNLI [18], a standard and freely available model whose three-way verdicts map directly onto the needs of layer V2. The answer-writing model, gpt-oss-120b served by Groq, was chosen because it is a large open-weight model with a free tier and fast responses.

Table 4.1 Models Used and Justification

Stage	Model	Size in memory	Reason for selection
Embeddings	all-MiniLM-L6-v2	about 90 MB	Small, fast on CPU, 384-dimensional vectors
Reranking	BAAI/bge-reranker-base	about 1.1 GB	Cross-encoder; better ordering of candidates
Entailment (V2)	roberta-large-mnli	about 1.4 GB	Standard three-way NLI; independent of the generator
Generation	openai/gpt-oss-120b on Groq	Runs on the provider's servers	Large open-weight model; free tier; fast
Generation (ablations)	gemini-flash-lite-latest	Runs on the provider's servers	Free quota for the batch experiments

4.5 Methodology

This section describes the three algorithms at the core of the system in more detail: the section-aware chunking algorithm, the reciprocal rank fusion used to merge the two retrieval lists, and the scoring rule that turns the verification checks into a decision. It also describes how the system routes questions between the verified path and the general-knowledge path. These algorithms are deliberately simple. Each of them can be explained in a few sentences and computed by hand for a small example, which makes the behaviour of the system predictable and its decisions easy to audit. Simplicity was a design goal in its own right, because a verification system whose decisions cannot be explained would only move the problem of trust from the language model to the verifier.

4.5.1 Section-Aware Chunking Algorithm

The chunking algorithm reads the document line by line and keeps track of the current chapter, section and clause. A line that matches the chapter pattern starts a new chapter; a line that matches the section pattern, a section number followed by a full stop and a heading, starts a new section; and a line that begins with a numbered sub-section or a lettered clause starts a new clause inside the current section. Text that follows a section heading before the first clause becomes the body of the section. When the whole document has been read, the algorithm emits one chunk for each section body and one

for each clause, each with its identifier, its text, its section summary and its metadata. If no section heading was found in the whole document, the algorithm reports that the document has no legal structure, and the document is passed to the paragraph fallback instead.

4.5.2 Reciprocal Rank Fusion

The dense and keyword searches each return a ranked list of chunks, but their scores are on different scales: cosine similarity lies between minus one and one, while BM25 scores are unbounded and depend on the collection. Reciprocal rank fusion [14] avoids comparing these scores by using only the rank of each chunk in each list. The fused score of a chunk d is computed with Equation 4.1, in which R is the set of ranked lists, w_r is the weight of list r, $rank_r(d)$ is the position of d in list r starting from one, and K is a smoothing constant set to 60. Both lists have a weight of 0.5. A chunk found by only one search therefore receives a score of about 0.008 at the top rank, while a chunk found at the top of both lists receives about 0.016, so agreement between the two searches is rewarded.

$$RRF(d) = \Sigma_{r \in R} w_r / (K + rank_r(d)) \quad (4.1)$$

4.5.3 Claim Score and Verification Confidence Score

Each claim receives a score from the results of layers V1 to V4. Two checks act as hard gates: if V1 finds that a cited chunk was not retrieved, or if V2 returns a CONTRADICTS verdict, the claim score is zero, because a fabricated citation or a refuting source cannot be compensated by other signals. Otherwise the claim score is the weighted mean of the components that ran, as shown in Equation 4.2, where e is 1 when V2 returns ENTAILS and 0 otherwise, f is the fraction of atomic facts entailed according to V3, c is the rate at which the claim recurs across resamples according to V4, and the weights w_e, w_f and w_c are 0.40, 0.35 and 0.25. A layer that did not run is removed from both the numerator and the denominator, so skipping a layer never lowers a score unfairly and the score always stays between zero and one.

$$s = (w_e \cdot e + w_f \cdot f + w_c \cdot c) / (w_e + w_f + w_c) \quad (4.2)$$

Layer V5 then computes the Verification Confidence Score of the whole answer as the mean of its N claim scores, as shown in Equation 4.3, and compares it with the threshold τ, which is set to 0.60. If the score is at least the threshold, the answer is shown with the Verified badge; otherwise the system abstains. The answer is also withheld when the model returned the abstention marker or when the parser found no claim that could be verified. The threshold of 0.60 is a provisional default that has not yet been calibrated on held-out data, and Chapter 6 discusses how this choice affects false refusals. The use of the mean also has a weakness, discussed in the error analysis: a weak claim can be

shown when it sits next to a strong one, and gating each claim on its own score is proposed as future work.

$$VCS = (1/N)\ \Sigma_{i=1..N}\ s_i\ ,\quad answer\ if\ VCS \geq \tau \tag{4.3}$$

4.5.4 Abstention and General-Knowledge Routing

Withholding an answer is the intended outcome when the documents do not support one, and the system distinguishes three kinds of response. A verified answer is shown with its badge, score and citations. An abstention tells the user that the indexed documents do not contain enough information, and it is produced either by the model's abstention marker or by a score below the threshold. A general-knowledge answer is produced only for questions about a general legal concept, such as "What is a non-disclosure agreement?", which are usually not defined in any indexed document. For these, a router sends the question to a separate path in which the model answers from its own knowledge, and the reply is labelled as general knowledge with no citations and no score. Any question that refers to the indexed documents, or names a specific Act, party or section, stays on the verified path.

4.6 Data Design

The only persistent store of the pipeline is the LanceDB table of chunks, described in Table 4.2, together with a folder of saved conversations used by the Ask page. Each row of the table represents one chunk. The vector column holds the 384-dimensional embedding, and the identifier column holds the chunk identifier that the language model must cite. The text column holds the chunk text exactly as it will be shown in the proof view, and the summary column holds the contextual summary used for embedding and reranking. The fingerprint column holds the SHA-256 hash of the source document, which is used to detect duplicates, and the metadata column holds the source identifier, the Act title, the chapter, the section number and title, the clause label and a readable section reference. Deleting a document removes every row with its source identifier, and the keyword index is rebuilt whenever the table version changes.

Table 4.2 Structure of a Chunk Row in the LanceDB Index

Field	Type	Description
vector	float[384]	Normalised embedding of summary and text
id / chunk_id	string	Citable identifier, for example ipc_theft::s379
text	string	Section body or clause text
contextual_summary	string	Section heading and first sentence
content_sha256	string	Fingerprint of the source document
source_id	string	Identifier of the uploaded document
act, chapter	string	Title of the Act and chapter name

Field	Type	Description
section, section_title, clause	string	Position of the chunk in the hierarchy
section_ref	string	Readable reference such as Section 7(a)

4.7 UML Diagrams

The Unified Modeling Language diagrams in this section describe the system from several points of view. The use case diagram shows what the user can do with the system and how the language model service takes part. The sequence diagram shows the order of messages exchanged between the components when a question is answered. The activity diagram shows the decision logic of the verification chain for a single answer. The class diagram shows the main classes of the implementation and the relationships between them. Finally, the data flow diagrams show how documents, questions, chunks and answers move between the user, the processes of the system and its data store. Together these diagrams complement the pipeline and architecture figures presented earlier in this chapter and give a complete picture of both the structure and the behaviour of the system.

4.7.1 Use Case Diagram

Fig. 4.4 shows the use case diagram. The primary actor is the user, who can upload documents, ask a legal question, search the index and view the evaluation results. Replacing a duplicate document extends the upload use case, because it happens only when an upload matches a document already in the index. Asking a question always includes the verification of its claims by the six-layer chain, and it is extended by two alternative outcomes: viewing a verified answer with its proof, or receiving an abstention when the documents do not support an answer. The Groq language model service appears as a secondary actor that takes part in answering a question, since it writes the cited answer from the retrieved passages. The diagram makes clear that verification is not an optional step that the user can skip but an integral part of every question that is asked.

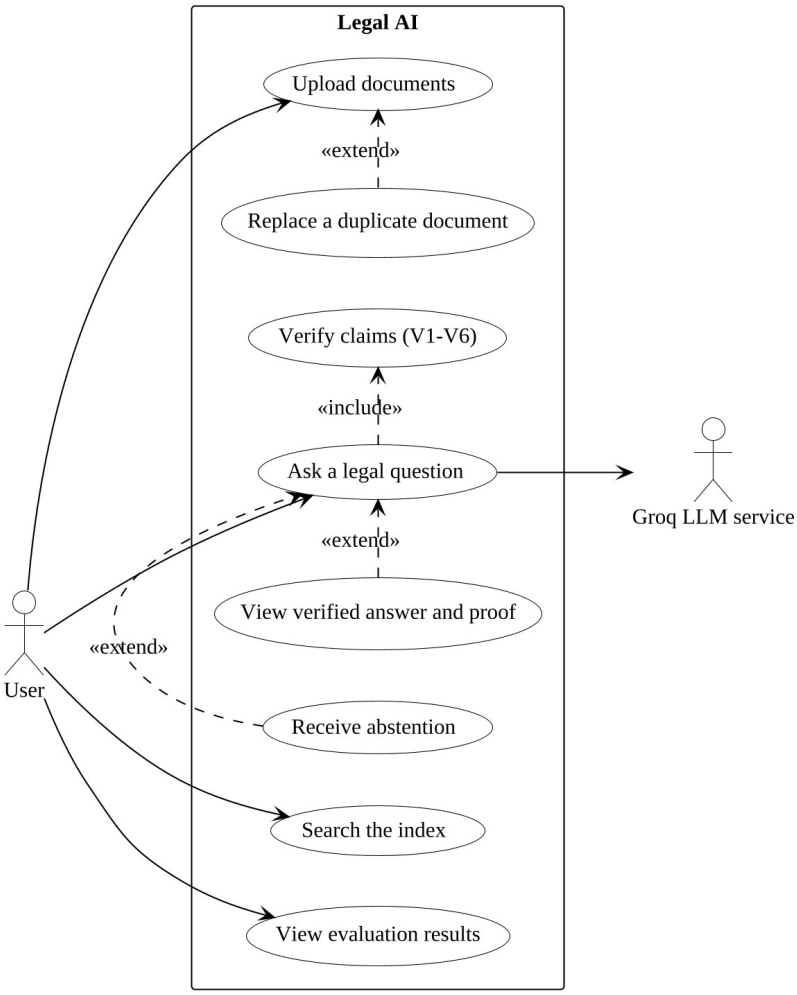


Fig. 4.4 Use case diagram of Legal AI

4.7.2 Sequence Diagram

Fig. 4.5 shows the sequence of messages when the user asks a question in the Ask page. The web application sends the question to the /query endpoint with a request for twelve passages. The API asks the hybrid retriever for a pool of one hundred candidates, and the retriever runs dense search and BM25 search against LanceDB, receives two ranked lists and merges them with reciprocal rank fusion. The cross-encoder then reranks the pool and returns the best twelve chunks. The API builds the prompt with these chunks and the citation rules and sends it to the Groq language model, which returns one claim per line with chunk identifiers. The API passes the claims to the verification chain, which returns the score, the decision and the proof. Finally the API returns either the answer or the abstention, with the proof, and the web application displays the badge, the citations and the proof view.

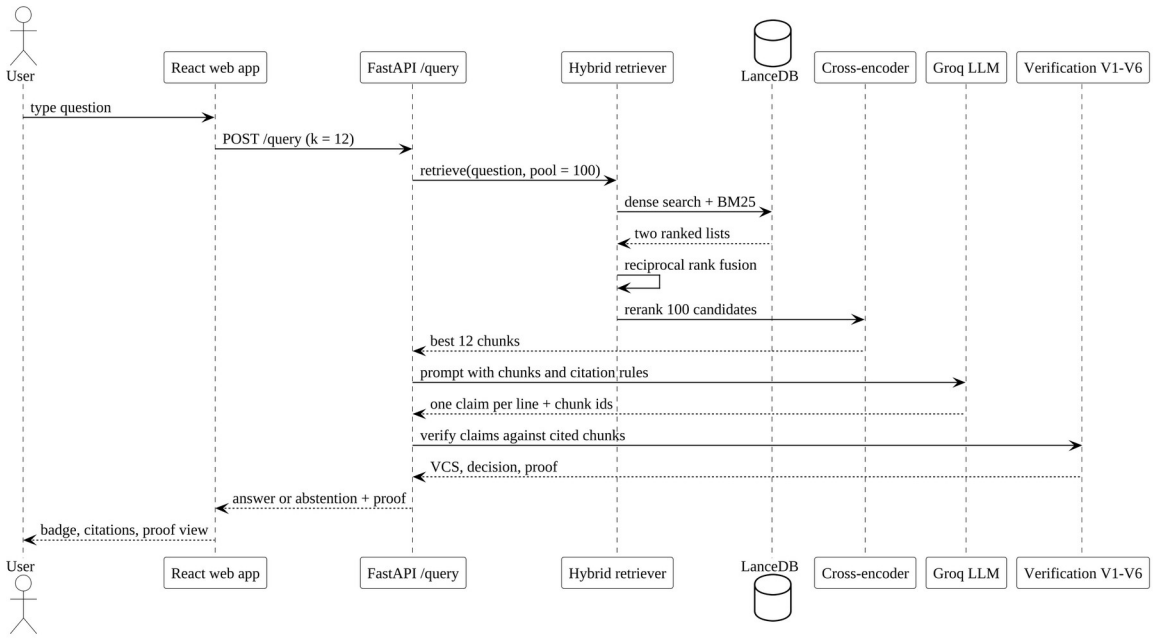


Fig. 4.5 Sequence diagram for answering a question

4.7.3 Activity Diagram

Fig. 4.6 shows the activity diagram of the verification and decision logic for one generated answer. If the answer is the abstention marker, the system shows an abstention immediately. Otherwise the answer is split into claims with their citations, and each claim is processed in turn. If its cited chunk was not retrieved, the claim score is zero. If the citation is valid, the inference model gives its verdict, and a contradiction again gives a score of zero. Otherwise the atomic facts are checked, the optional consistency check is applied, and the weighted claim score is computed. When all claims have been processed, the Verification Confidence Score is computed as the mean of the claim scores. If it is at least 0.60, the answer is shown as verified and the proof object is attached; otherwise the system abstains and states that the answer is not supported by the documents.

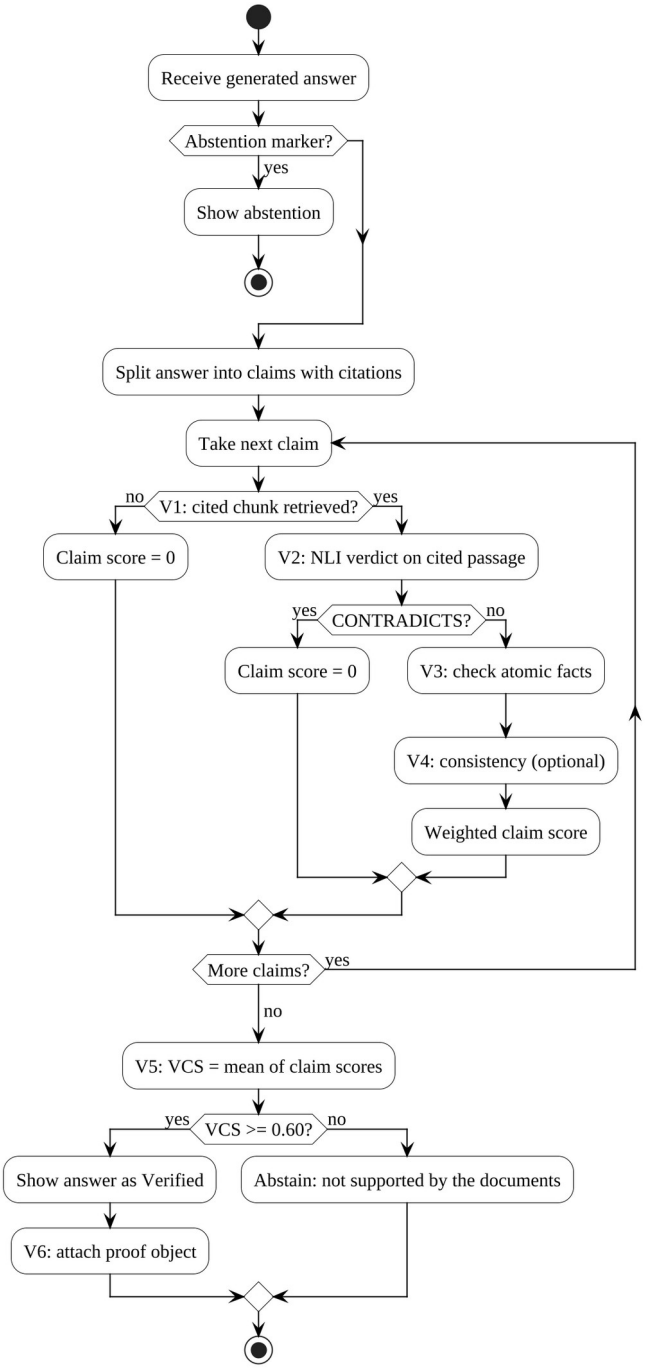


Fig. 4.6 Activity diagram of the verification chain and the decision rule

4.7.4 Class Diagram

Fig. 4.7 shows the main classes of the implementation. A ChunkRecord holds the identifier, text, contextual summary and metadata of one chunk, and a ChunkStore manages the LanceDB table that holds many chunk records, with operations to embed chunks, write rows, find duplicates and delete a source document. The FusedRetriever combines the dense and keyword retrievers over the store with weighted reciprocal rank fusion, and the ScoredCrossEncoderReranker reranks its output with the bge-reranker-base model while keeping the relevance scores visible. On the verification side,

RobertaMNLI classifies premise and hypothesis pairs for layer V2, VCSConfig holds the weights, the threshold and the switches used to disable individual layers in the ablations, VCSResult holds the score, the decision and the per-claim results, and the proof object is built from the verification result and returned to the interface.

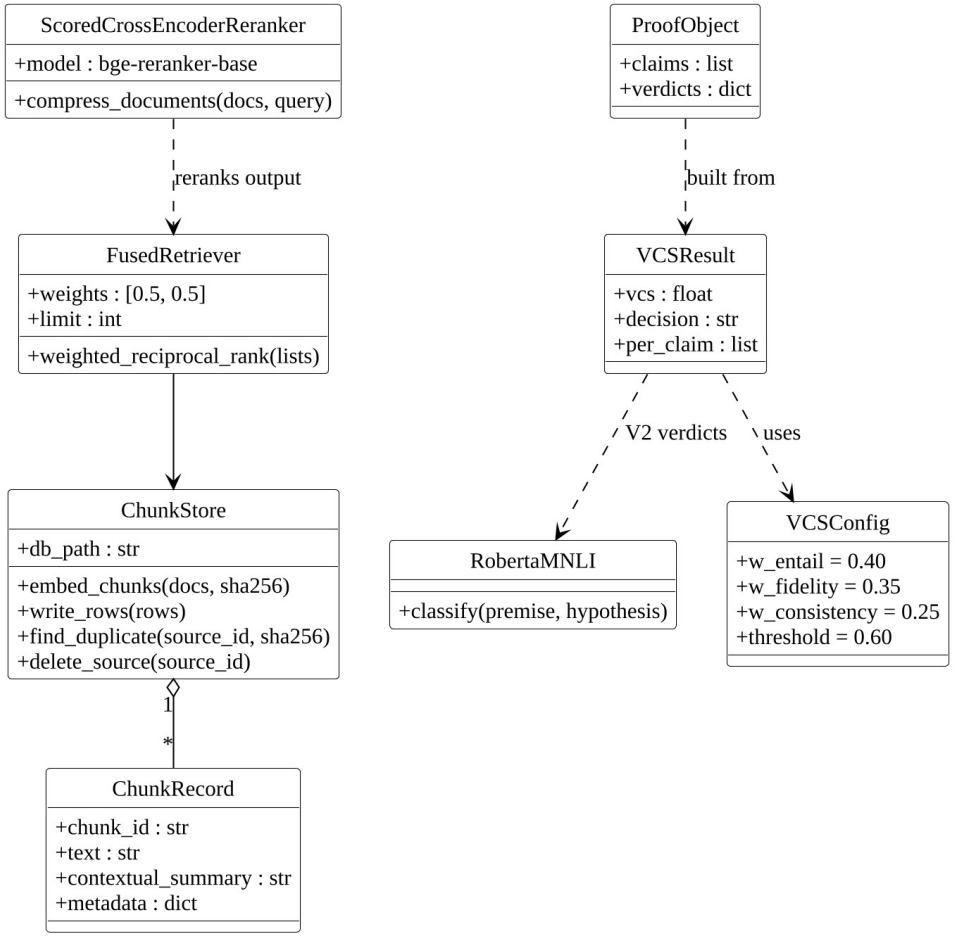


Fig. 4.7 Class diagram of the main components

4.7.5 Data Flow Diagrams

Fig. 4.8 shows the level 0 data flow diagram, also called the context diagram, in which the whole system is a single process. The user sends documents and questions to the system and receives verified answers or abstentions together with their proofs, while the system sends prompts containing the cited chunks to the language model service and receives claims with chunk identifiers in return. Fig. 4.9 shows the level 1 data flow diagram, which opens the system into five processes. Process 1.0 ingests and chunks a document, process 2.0 embeds the chunks and writes them to the LanceDB data store D1, process 3.0 retrieves and reranks candidate chunks for a question, process 4.0 generates the cited answer with the help of the language model, and process 5.0 verifies the claims, decides whether to show the answer and returns the result and its proof to the user.

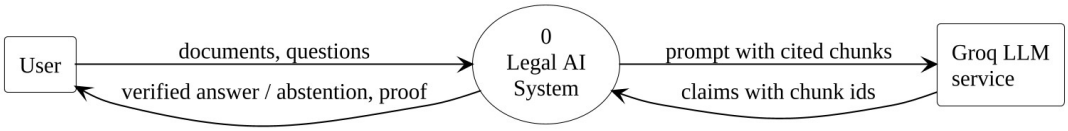


Fig. 4.8 Level 0 data flow diagram (context diagram)

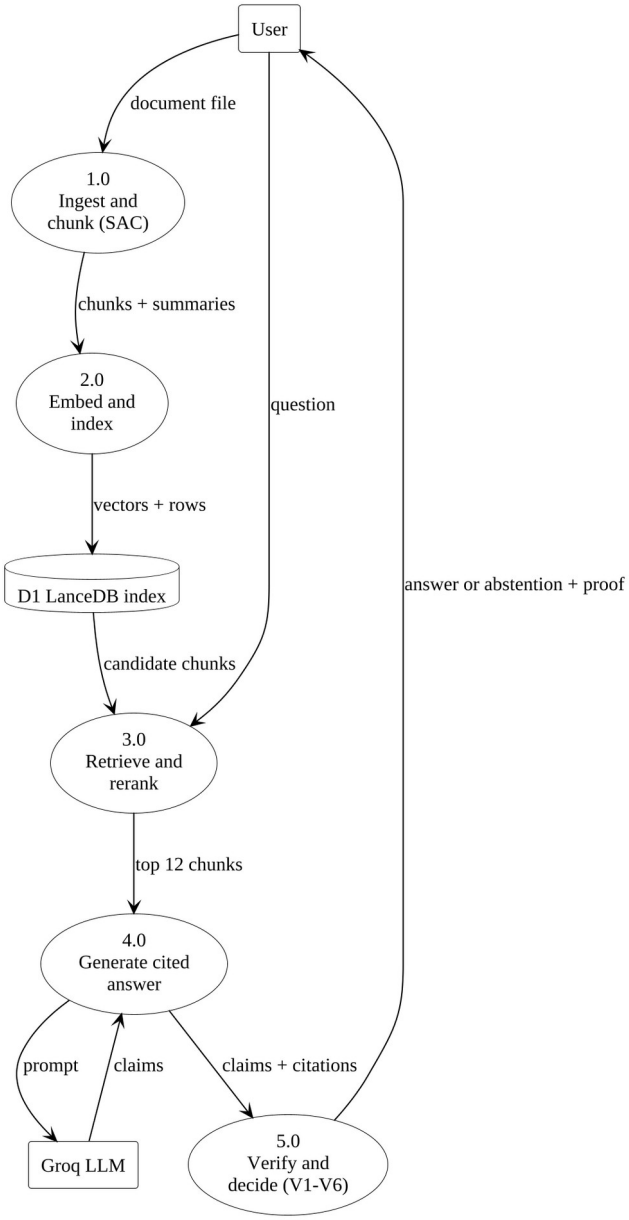


Fig. 4.9 Level 1 data flow diagram

4.8 Comparison with Existing Approaches

Table 4.3 compares the design of Legal AI with the approaches discussed in Chapters 2 and 3: a general-purpose chatbot used directly, a standard retrieval-augmented system with fixed-size chunks, summary-augmented chunking as proposed by Reuter et al. [10], and a legal retrieval-augmented pipeline with a post-generation citation audit in the style of Figueiredo et al. [22]. The comparison is based on the design features

described in each case and not on a shared benchmark, since the systems were evaluated on different data. It shows that the distinctive features of Legal AI are the combination of structure-preserving chunking with hybrid retrieval, a citation for every claim, an independent entailment check for every claim, a single score that gates the answer, and a proof object that the user can open, together with a clearly labelled path for general-knowledge questions that keeps unsourced answers separate from verified ones.

Table 4.3 Design Comparison with Existing Approaches

Feature	Chatbot	Standard RAG	SAC [10]	Citation audit [22]	Legal AI
Answers from uploaded documents	No	Yes	Yes	Yes	Yes
Structure-aware chunks	No	No	Yes	Not stated	Yes
Hybrid search with reranking	No	Varies	Varies	Varies	Yes
Citation for every claim	No	Varies	Varies	Yes	Yes
Entailment check per claim	No	No	No	Varies	Yes
Score that gates the answer	No	No	No	No	Yes
Per-claim proof for the user	No	No	No	Partial	Yes

CHAPTER 5
IMPLEMENTATION

5.1 Experimental Setup

The system was implemented in Python 3.11 for the back end and JavaScript for the front end, and Table 5.1 summarises the setup used for development and for the experiments. The pipeline was composed with LangChain, the index was stored in LanceDB, and the local models were run with sentence-transformers and Hugging Face Transformers on the CPU. For the ablation experiments and the batch test on real contracts, answers were generated with gemini-flash-lite-latest because of its free quota, while the end-to-end test through the web interface used openai/gpt-oss-120b served by Groq on the LangChain back end. Each answer in the ablations was generated once and then replayed through every arm of the experiment, so that the arms differ only in the layer that was removed and not in the randomness of the model. Layer V4 was run on a five-question subset with two resamples each, to stay within the free quota of the model service.

Table 5.1 Experimental and Development Setup

Item	Setting
Programming languages	Python 3.11 (back end), JavaScript (front end)
Frameworks and libraries	LangChain, LanceDB, sentence-transformers, Transformers, FastAPI, React, Vite
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Reranker	BAAI/bge-reranker-base
Inference model (V2)	roberta-large-mnli
Generator (ablations, batch test)	gemini-flash-lite-latest
Generator (web interface, end-to-end test)	openai/gpt-oss-120b on Groq
Testing tools	pytest (unit and API tests), Playwright (browser automation)
Version control	Git and GitHub

5.2 Project Structure

The repository is organised so that the pipeline, the user interface, the data and the experiments are kept apart, and Table 5.2 lists its main folders. The src folder contains the plain-Python implementation of every stage, including ingestion, chunking, embedding, indexing, retrieval, generation, verification, evaluation and the original API, and the verification chain and the structure parser in this folder are also used by the

LangChain version. The lc folder contains the LangChain implementation of loading, retrieval and generation and the API used by the web application, including the hosted entry point that serves the API and the built web application from one server. The web folder contains the React application. The data folder holds the synthetic statutes, the evaluation question sets and the Indian criminal law pack, and the experiments folder holds the configuration and result files of every logged run, from which every number in this report is taken.

Table 5.2 Main Folders of the Project Repository

Folder	Contents
src/	Plain-Python pipeline: ingestion, chunking, embedding, indexing, retrieval, generation, verification, evaluation, API
lc/	LangChain pipeline and the FastAPI application used by the web app; hosted entry point
web/	React single-page application (Home, Ingest, Ask, Search, Evaluation)
data/	Synthetic statutes, evaluation sets, Indian criminal law pack and its questions
experiments/	Configuration and result files of every logged experiment
scripts/	Building the crime pack, loading a folder, batch question checks
docs/	Architecture, phase notes, paper, guides, demo questions and videos
.devcontainer/, deploy/	Configuration for GitHub Codespaces and Docker hosting

5.3 Datasets Used

Three kinds of documents were used in this project, each for a different purpose. For the controlled experiments we wrote five synthetic statutes covering urban tenancy, data privacy, consumer protection, small business licensing and environmental waste management. Each follows an Act, Chapter, Section and Clause hierarchy, and together with them we wrote ten questions whose answers were labelled at clause level, fourteen gold clauses in all. Writing the statutes ourselves meant that the correct answer to every question was known exactly, which is necessary to measure retrieval precision and recall without disagreement. The disadvantage of this choice is that the statutes are short and regular, which makes the retrieval task easy, and this limitation is discussed together with the retrieval results in Chapter 6 and again among the limitations in Chapter 7.

To test the system on documents that the team did not write, we used real contracts from public research datasets: non-disclosure agreements from ContractNLI [27], a non-competition agreement from CUAD [28], and agreements from MAUD [29]. These

documents have no statutory structure, so they exercise the paragraph fallback, and they were used in the batch test and the end-to-end test but not in the scored ablations. For demonstrations with Indian law, we built a pack of twenty-seven files of criminal law covering 429 sections of the Indian Penal Code, the Code of Criminal Procedure, the Indian Evidence Act and the Negotiable Instruments Act, which produce 493 chunks when indexed. The text comes from a public dataset of Indian Acts on GitHub [30], whose content corresponds to the Acts published by the Government of India on India Code [31]; a script converts it into the format read by the structure parser.

5.4 Key Parameters

Table 5.3 lists the parameters that control the behaviour of the system. Most of them were fixed during design and left unchanged in all experiments, so that the results can be compared. The number of passages given to the model in the Ask page, twelve, balances the chance of including the right passage against the length of the prompt and the cost of verifying more citations. The candidate pool of one hundred reflects the lesson, learned while testing on real contracts, that a shallow pool prevents the reranker from finding correct clauses ranked low by the first stage. The verification weights give the largest share to the entailment verdict, followed by atomic fact fidelity and consistency. The threshold of 0.60 is a provisional default rather than a calibrated value, and the paragraph and clause sizes keep every chunk within the input limits of the embedding and inference models.

Table 5.3 Key Implementation Parameters

Parameter	Value	Where used
Embedding dimension	384	Chunk and question vectors
Passages given to the model (Ask)	k = 12	Generation prompt
Candidate pool before reranking (Ask)	max(8k, 100) = 100	Hybrid retrieval
Results per method (Search page)	k = 5	Retrieval inspection
Experiments: k and pool	k = 5, n = 50	Retrieval and verification ablations
RRF constant and weights	K = 60; 0.5 and 0.5	Fusion of dense and BM25 lists
Claim weights	0.40 / 0.35 / 0.25	Entailment / fidelity / consistency
Abstention threshold	$\tau = 0.60$	Decision rule (V5)
Paragraph fallback size	at most 700 characters	Documents without structure
Clause size in the crime pack	at most 180 words	Keeps chunks within model limits

5.5 Key Code Snippets

This section presents the key functions of the four main parts of the pipeline, taken from the source code in the project repository: text extraction, the query pipeline, the comparison and fusion of retrieval results, and the verification rules together with the orchestration of the six layers. Only the functions themselves are shown, not complete programs, and some functions are abridged, with comments and error handling removed and omitted lines marked with an ellipsis, so that the central logic fits on the page. Each excerpt is followed by an explanation of what it does and why it matters for the behaviour of the system. The complete source files, with their tests and documentation, are available in the GitHub repository described in Appendix A, where each function can be read in full together with the modules it calls.

5.5.1 Text Extraction

```
def load_pdf(path: str | Path) -> str:
    """Load a PDF document and extract its text, page by page, via pypdf."""
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError(f"No such file: {path}")
    reader = PdfReader(str(path))
    pages_text = [page.extract_text() or "" for page in reader.pages]
    return "\n".join(pages_text)

def load_document(path: str | Path) -> str:
    """Dispatch to the right loader based on file extension."""
    path = Path(path)
    suffix = path.suffix.lower()
    try:
        loader = _LOADERS[suffix]      # .txt, .pdf
    except KeyError as exc:
        raise ValueError(f"No loader registered for {suffix!r}") from exc
    return loader(path)
```

```
def ingest_file_with_text(path: str | Path) -> tuple[Document, str]:
    """Load (.txt or .pdf), clean, and parse; return the parsed Document and
    the cleaned text, which fallback chunking needs when the document has no
    legal structure to parse."""
    path = Path(path)
    raw = load_document(path)          # pypdf for PDF, plain read for .txt
    cleaned = clean_text(raw)
    document = parse_document(cleaned, source_id=path.stem.strip())
    return document, cleaned
```

This function is the entry point of text extraction. It reads an uploaded file with the loader for its type, which uses the pypdf library for PDF files and a plain read for text files, and then cleans the text by normalising whitespace and removing artefacts of extraction while keeping the line breaks that mark headings. The cleaned text is passed

to the structure parser, which recognises chapters, sections and clauses and returns a document object with its legal hierarchy. The function also returns the cleaned text itself, because a document without any legal structure, such as a contract, is split by the paragraph fallback instead. The source identifier is taken from the file name with surrounding spaces removed, a fix added after testing showed that a stray space in a file name broke every citation identifier of that document.

```
def parse_document(cleaned_text: str, source_id: str) -> Document:
    """Parse a cleaned statute into its Act/Chapter/Section/Clause hierarchy."""
    lines = [line.strip() for line in cleaned_text.split("\n")]
    chapters, current_chapter, current_section = [], None, None
    ...
    for line in lines:
        if not line:
            continue
        m = _CHAPTER_RE.match(line)
        if m:
            flush_section()
            current_chapter = Chapter(number=m.group(1), name=m.group(2).strip())
            chapters.append(current_chapter)
            continue
        m = _SECTION_RE.match(line)          # "Section 379. Punishment..."
        if m:
            flush_section()
            current_section = Section(number=m.group(1), title=m.group(2).strip())
            continue
        m = _LETTERED_CLAUSE_RE.match(line)  # "Clause (a): ..."
        if m and current_section is not None:
            current_section.clauses.append(
                Clause(label=m.group(1).lower(), text=m.group(2).strip()))
            continue
        m = _NUMBERED_CLAUSE_RE.match(line)  # "(1) ..."
        if m and current_section is not None:
            current_section.clauses.append(
                Clause(label=m.group(1), text=m.group(2).strip()))
            continue
        if current_section is not None:      # body text of the section
            current_section.body_lines.append(line)
    flush_section()
    return Document(source_id=source_id, title=title or source_id, ...)
```

The structure parser reads the cleaned text line by line and builds the legal hierarchy that summary-augmented chunking needs. Regular expressions recognise each kind of line: a chapter heading starts a new chapter, a line such as "Section 379. Punishment for theft." starts a new section, and a line that begins with a lettered clause or a numbered sub-section adds a clause to the current section. Any other line is treated as part of the body of the current section. Whenever a new section or chapter begins, the helper flush_section stores the section that was being built, so no text is lost at the boundaries. The section pattern accepts letter suffixes such as 498A and 13B, which appear in Indian statutes. If the whole document produces no section at all, the caller knows that

the document has no legal structure and passes its cleaned text to the paragraph fallback instead.

5.5.2 Query Pipeline

```
def build_query_chain(retriever, llm, nli) -> Runnable:
    """Input: {question, self_consistency, allow_general_knowledge}."""
    answer = answer_chain(llm)          # prompt | llm | citation parser
    ...
    return (
        RunnablePassthrough.assign(context=itemgetter("question") | retriever)
        | RunnablePassthrough.assign(prompt=RunnableLambda(_prompt_messages))
        | RunnablePassthrough.assign(parsed=_guard(answer, "generate"))
        | RunnablePassthrough.assign(resamples=_guard(RunnableLambda(_resamples),
                                                      "resample"))
        | RunnablePassthrough.assign(verification=verification_step(nli))
        | RunnableBranch(
            (wants_general_knowledge,
             RunnablePassthrough.assign(general_text=RunnableLambda(_general))),
            RunnablePassthrough.assign(general_text=lambda _: None),
        )
    )
```

This function builds the complete question-answering pipeline as a single LangChain chain. Each step adds its result to a shared state without removing what came before: the retriever adds the context chunks, the prompt step adds the exact messages sent to the model, the answer step adds the parsed claims and citations, the optional resampling step adds extra answers for the consistency check, and the verification step adds the score, the decision and the proof object. The final branch sends a question about a general legal concept to the general-knowledge path, which produces a labelled answer without citations, and leaves every other question on the verified path. Because the state keeps every intermediate result, the API can return a full trace of how an answer was produced, which was essential for finding the defects described in Chapter 6.

```
def answer_chain(llm) -> Runnable:
    """{question, context} -> ParsedAnswer: prompt | llm | str | parser."""
    return (
        RunnableLambda(prompt_inputs, name="prompt_inputs")
        | ANSWER_PROMPT
        | as_chat_runnable(llm)
        | StrOutputParser()
        | CitationOutputParser()
    )

class CitationOutputParser(BaseOutputParser[ParsedAnswer]):
    def parse(self, text: str) -> ParsedAnswer:
        if text.strip().startswith(f"{ABSTENTION_MARKER}:"):
            return ParsedAnswer(raw_text=text, abstained=True)
        claims, malformed = [], []
        for raw_line in text.splitlines():
            line = raw_line.strip()
            if not line:
                continue
            ids = self.citations(line)          # chunk ids in [brackets]
            if ids:
                text = _CITATION_RE.sub("", line).strip()
                claims.append(ParsedClaim(text, ids))
            else:
                malformed.append(line)          # a sentence with no citation
        return ParsedAnswer(raw_text=text, abstained=False,
                            claims=claims, malformed_lines=malformed)
```

The answer step itself is a small chain of four parts. The prompt step fills the citation-forced prompt with the question and the retrieved chunks, the model wraps whichever language model provider is configured, the string parser takes the text of the reply, and the citation parser turns that text into a structured answer. The parser first checks for the abstention marker, in which case the answer is recorded as an abstention with no claims. Otherwise it reads the reply line by line, extracts the chunk identifiers written in square brackets at the end of each line, and stores every line that has at least one identifier as a claim with its citations. A line without any citation is kept separately as a malformed line and is never shown as a claim, because a factual sentence without a source cannot be verified by the layers that follow.

5.5.3 Retrieval Comparison and Fusion

```
def weighted_reciprocal_rank(self, doc_lists):
    scores, first = {}, {}
    for doc_list, weight in zip(doc_lists, self.weights):   # dense, BM25
        for rank, doc in enumerate(doc_list, start=1):
            key = doc.metadata[self.id_key]
            scores[key] = scores.get(key, 0.0) + weight / (rank + self.c)
            first.setdefault(key, doc)
    ranked = sorted(scores, key=lambda key: scores[key],
                    reverse=True)[: self.limit]
    return [_copy(first[key], _relevance_score=scores[key]) for key in ranked]

def query_retriever(store, k, n, reranker=None):
    if reranker is None:
        return hybrid_retriever(store, k)
    return ContextualCompressionRetriever(
        base_compressor=as_compressor(reranker, top_n=k),
        base_retriever=hybrid_retriever(store, n))
```

The first function compares and merges the ranked lists returned by dense search and BM25 keyword search. For every list it walks through the chunks in order and adds the weight of the list divided by the rank plus a constant to the score of each chunk, which is Equation 4.1, so a chunk found near the top of both lists collects two contributions and moves up. The chunks are then sorted by their fused score and the best ones are returned with the score attached, so the Search page and the proof view can display it. The second function puts the stages together: without a reranker it returns the fused top k directly, and with a reranker it first fetches a larger pool of n fused candidates and then lets the cross-encoder keep the best k of them, which is how the Ask page turns a pool of one hundred candidates into twelve passages.

```
def bm25_retriever(store: ChunkStore, k: int) -> BaseRetriever:
    key = (store.db_path, store.version())
    if key not in _BM25_CACHE:          # rebuilt after an ingest or delete
        docs = store.all_documents()
        _BM25_CACHE.clear()
        _BM25_CACHE[key] = (
            ScoredBM25Retriever.from_documents(
                docs, preprocess_func=lance_tokenize)
            if docs else _EmptyRetriever())
    base = _BM25_CACHE[key]
    return base.model_copy(update={"k": k})

def dense_retriever(store: ChunkStore, k: int) -> DenseRetriever:
    return DenseRetriever(store=store, k=k)

def hybrid_retriever(store: ChunkStore, k: int) -> FusedRetriever:
    """Dense + BM25, each fetching k, RRF-fused and truncated to k."""
    return FusedRetriever(
        retrievers=[dense_retriever(store, k), bm25_retriever(store, k)],
        weights=[0.5, 0.5],
        c=59,
        id_key="chunk_id",
        limit=k,
    )
```

These functions create the two retrieval methods that are compared and combined. The keyword retriever builds a BM25 index over the text of every chunk in the LanceDB table, using the same simple tokenizer as LanceDB, which lowercases the text and splits it on characters that are not letters or digits. Because building this index takes time, it is kept in a cache keyed by the database path and the version of the table, so it is rebuilt only after a document is ingested or deleted rather than for every question. The dense retriever searches the embedding vectors stored in LanceDB. The hybrid retriever joins the two with equal weights of 0.5 and the fusion constant, and returns the best k fused chunks. The Search page calls the dense, keyword and hybrid retrievers separately for the same query, so that their results can be compared side by side.

5.5.4 Verification Rules and Orchestration

```
def check_citation_existence(answer: Answer) -> list[ClaimCitationResult]:
    """Check each claim's citations against the retrieved context ids."""
    context_ids = set(answer.context_chunk_ids)
    results: list[ClaimCitationResult] = []
    for claim in answer.claims:
        existing = [cid for cid in claim.cited_chunk_ids
                    if cid in context_ids]
        missing = [cid for cid in claim.cited_chunk_ids
                   if cid not in context_ids]
        results.append(ClaimCitationResult(
            claim_text=claim.text,
            cited_chunk_ids=list(claim.cited_chunk_ids),
            existing_ids=existing,
            missing_ids=missing))
    return results
```

Layer V1 is the simplest rule of the verification chain and the first one to run. It collects the identifiers of all the chunks that were actually retrieved for the question, and then, for every claim, separates the identifiers the claim cites into those that exist among the retrieved chunks and those that do not. A claim passes only when it cites at least one identifier and every cited identifier was retrieved. A claim that cites an identifier that was never retrieved has, by definition, a fabricated citation, so its score is set to zero by the scoring rules shown next. The missing identifiers are also kept in the result, which allows the proof view to name exactly which citation was invented. This is the distinct contribution of V1 observed in the ablation, since the other layers also score such a claim at zero but cannot say why.

```
def _claim_vcs(citation_ok, entailment, fidelity, consistency, config):
    if config.enable_v1 and not citation_ok:
        return 0.0, "citation"                    # hard gate V1
    if config.enable_v2 and entailment == Entailment.CONTRADICTS:
        return 0.0, "contradiction"               # hard gate V2
    components = [(1.0 if entailment == Entailment.ENTAILS else 0.0,
                   config.w_entail), (fidelity, config.w_fidelity)]
    if consistency is not None:
        components.append((consistency, config.w_consistency))
    total_weight = sum(w for _, w in components)
    return sum(c * w for c, w in components) / total_weight, None

def verify_answer(answer, context_chunks, nli, resamples=None, config=None):
    v1 = check_citation_existence(answer)                      # V1
    v2 = check_entailment_all(answer.claims, context_chunks, nli)  # V2
    v3 = check_fidelity_all(answer.claims, context_chunks, nli)    # V3
    v4 = self_consistency(answer, resamples) if resamples else None  # V4
    vcs_result = aggregate_vcs(...)             # V5: claim scores, gate 0.60
    proof = build_proof(answer.query, answer.raw_text, context_chunks,
                        v1=v1, v2=v2, v3=v3, v4=v4, vcs_result=vcs_result)  # V6
    return VerificationResult(vcs_result, proof)
```

The first function contains the rules that turn the checks of one claim into a score, which is Equation 4.2. An invalid citation or a contradiction sets the score to zero and records which gate failed, so the proof view can show the reason. Otherwise the function takes the weighted mean of the entailment verdict, the fraction of atomic facts entailed and, when it was measured, the consistency of the claim across resamples, leaving out any layer that did not run. The second function orchestrates the six layers in order, since later layers use the results of earlier ones: it checks the citations, runs the inference model on every claim, checks the atomic facts, measures consistency when resampled answers are supplied, combines everything into the Verification Confidence Score and the decision with the threshold of 0.60, and finally builds the proof object that the web interface shows when a citation is clicked.

5.6 Deployment and User Interface

For everyday use, the system runs on a single computer. A batch file in the repository starts the FastAPI back end on port 8000 and the React development server on port 5173 in two windows and then opens the browser. The index is stored in a local folder that is not part of the repository, so on a new machine it starts empty and the documents must be uploaded once, either on the Ingest page, where many files can be selected together as shown in Fig. 5.1, or with a script that loads a whole folder through the API. For the demonstration, the twenty-seven files of the Indian criminal law pack can be loaded in a single step, after which the Home page shows 27 documents and 493 chunks, and the questions listed in the appendix can be asked on the Ask page. The Search page, shown in Fig. 5.2, lets the audience see what each retrieval method finds before any answer is generated.

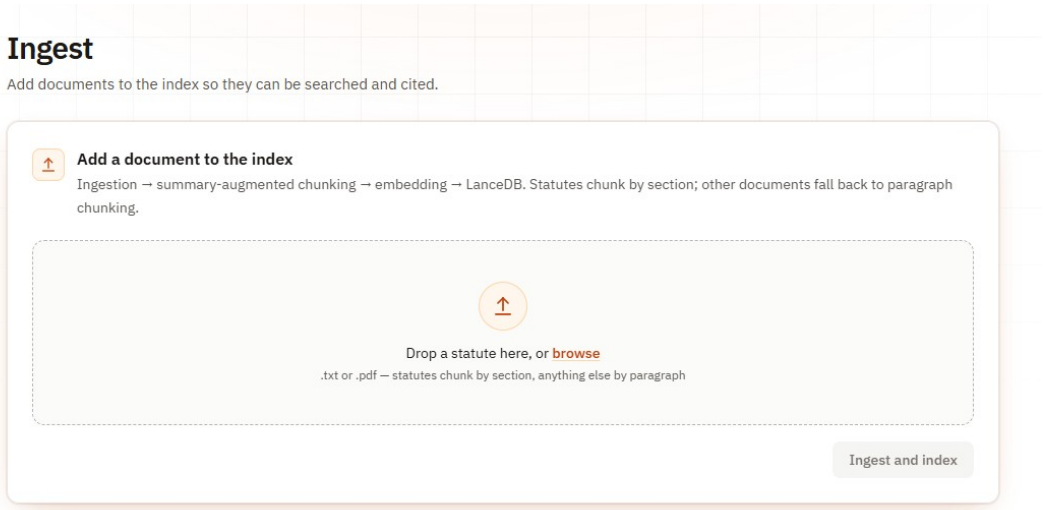


Fig. 5.1 The Ingest page, where documents are uploaded and indexed

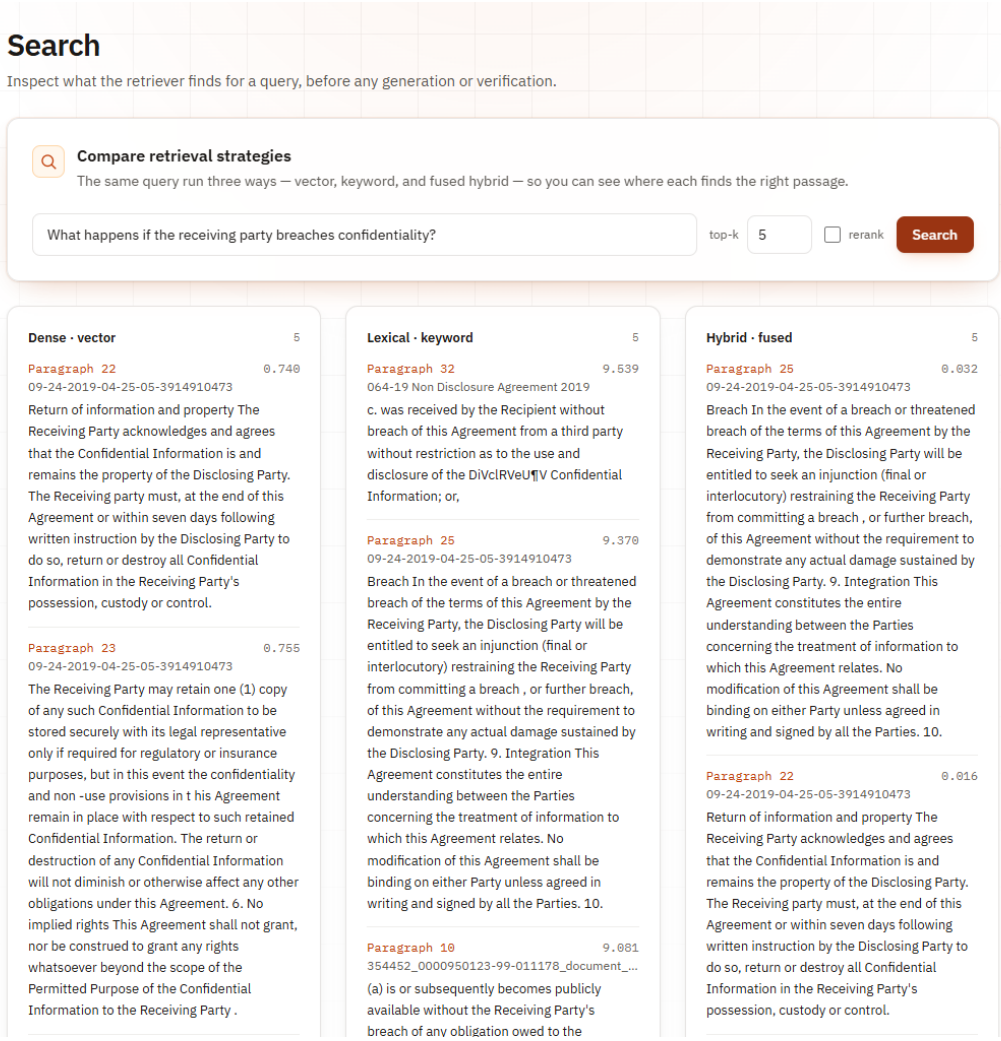


Fig. 5.2 The Search page, comparing dense, keyword and hybrid retrieval for one query

For online access, the project includes a hosted entry point that serves the API under the /api path and the built web application at the root of the same address, so that the API routes never collide with the page addresses of the web application. When this server starts with an empty index, it loads the criminal law pack in the background and loads the models once, so that a fresh installation can answer questions straight away. Two hosting configurations are provided. A development container configuration allows the complete application to be started in GitHub Codespaces, which provides a cloud machine with enough memory for the local models and a public address for the application at no cost within the free monthly allowance. A Dockerfile packages the same application as a single container for any host that offers about four gigabytes of memory.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Retrieval was scored for each question against the labelled gold clauses and averaged over the question set. Precision is the fraction of retrieved chunks that are relevant, recall is the fraction of relevant chunks that were retrieved, and the F1 score is their harmonic mean, as shown in Equation 6.1. The retrieval rate is the share of questions for which at least one gold clause appears among the top k results. Since summary-augmented chunks and paragraph chunks carry different identifiers, relevance was matched on the text of the chunk rather than on its identifier, which scores both chunking strategies by the same rule. We also recorded the mean number of characters of context passed to the generator per question, because a chunking strategy that finds the same evidence with less text gives the model a smaller prompt and gives the verifier fewer passages to check.

$$F1 = 2 \cdot P \cdot R / (P + R) \tag{6.1}$$

For verification, three kinds of measurement were used. On clean answers, we report the mean Verification Confidence Score and the number of questions answered and refused, which shows how cautious the system is when nothing has been corrupted. On deliberately corrupted answers, we report how many known-bad claims reached the user, which is the most direct measure of whether the system keeps wrong claims away. We also report weak claims, defined as claims that the full chain scores below the threshold of 0.60 but that are nevertheless shown because they sit in an answer that is shown. For the end-to-end test, each answer was compared with an expected answer written before the test, and labelled as correct and verified, wrong and verified, correct abstention, false abstention, general knowledge correctly labelled, or another outcome. Response time was measured per question.

6.2 Testing Strategy

Testing was carried out at four levels. At the unit level, every module has automated tests written with pytest, covering the structure parser, the chunkers, the duplicate check, the retrievers, the prompt and parser, each verification layer and the score computation. At the API level, tests drive the FastAPI application with lightweight stand-in models, so that uploading, retrieving, querying, deleting and conversation handling can be checked quickly and without network access. In the last run before this report, 355 automated tests passed across the plain-Python and LangChain suites, with the remaining tests skipped because they require the full models. At the experiment

level, the retrieval and verification ablations were run from stored configurations, and every result was written to files in the experiments folder of the repository, from which the tables in this chapter were produced.

At the system level, the end-to-end test used the real web interface, as a person would. Every upload, question and click was a real browser action driven by Playwright, against the LangChain back end with a fresh index. Before anything was indexed, the team read the documents and wrote fifteen questions, each with its expected behaviour, expected answer and source passage, and committed them to the repository so they could not be adjusted after seeing the results. The questions covered six categories: five about a specific fact, two comparing documents, three that the documents cannot answer, two about general legal concepts, two with a false premise, and one paraphrase that uses none of the keywords of its source. In addition, the batch test on ten real non-disclosure agreements and the scripted check of the Indian criminal law pack exercised the system on larger document sets.

6.3 Functional Test Results

Table 6.1 summarises the functional tests carried out through the web interface and the API, with the expected and the observed behaviour. All of these behaviours were observed during the end-to-end test or during the testing of the multi-document upload and the criminal law pack. They confirm that the main functions of the system work as specified: documents are indexed with the right chunking strategy, duplicates are refused and can be replaced, deleted documents stop being used, paraphrased questions find the right provision, unanswerable and false-premise questions are refused, general concept questions are labelled correctly, and corrupted citations never reach the user as verified claims. The table does not show the quality of every answer, which is measured separately in the quantitative results that follow, but it shows that each function of the system behaves as designed when it is used through the interface.

Table 6.1 Functional Test Cases

ID	Test	Expected behaviour	Observed	Status
T1	Upload a statute	Chunked by section with identifiers	23 chunks from the tenancy act, by section	Pass
T2	Upload the 27-file crime pack at once	All files indexed in one action	27 documents, 493 chunks	Pass
T3	Upload the same file again	Refused as a duplicate	Duplicate warning, nothing added	Pass
T4	Replace a duplicate	Old copy removed, new copy indexed	Same 23 chunks restored	Pass
T5	Delete a document, then ask about it	Question abstains	Abstained	Pass

ID	Test	Expected behaviour	Observed	Status
T6	Paraphrased question without keywords	Answer from the right clause	Verified, VCS 1.00, eviction clause	Pass
T7	Question the documents cannot answer	Abstention	Two of three abstained; one misrouted	Partial
T8	Question with a false premise	Abstention	Both abstained	Pass
T9	General concept question	Labelled general knowledge	Both labelled, no citations	Pass
T10	Fabricated citation in an answer	Not shown as verified	0 of 14 shown	Pass
T11	Upload an unsupported file type	Rejected with a reason	Rejected; other files indexed	Pass

6.4 Quantitative Results

This section presents the measured results of the four experiments: the retrieval and chunking ablation, the verification ablation, the end-to-end test through the interface, and the comparison between the LangChain and plain-Python implementations of the pipeline. All numbers are taken from the result files logged in the experiments folder of the repository. The question sets are small, ten labelled questions for the ablations and fifteen for the end-to-end test, so the results should be read as evidence about how the design behaves rather than as precise estimates of accuracy on legal questions in general. Where a result is limited by the data, for example because the question set is too easy to separate two retrieval settings, this is stated next to the result rather than left for the reader to discover, and the limitations are gathered again in Chapter 7.

6.4.1 Retrieval and Chunking

Table 6.2 shows the retrieval results of the eight configurations formed by two chunking strategies, two retrieval methods and reranking switched on or off, over ten questions with k equal to five and a candidate pool of fifty. Summary-augmented chunking reached a precision of 0.280 and an F1 score of 0.417 in three of the four retrieval settings, against 0.200 and 0.333 for paragraph chunking in every setting, as also shown in Fig. 6.1. Recall was at or near 1.000 throughout, so the difference comes from precision. Part of this difference is granularity, since precision at a fixed k favours smaller chunks. The more useful effect for the generator is shown in the last column: summary-augmented chunking passed about 550 characters of context per question, against 1,500 to 1,800 for paragraph chunks, so the model received about a third of the text with no loss of recall.

Table 6.2 Retrieval Results over 10 Questions (k = 5, n = 50); Ctx is the Mean Characters of Context per Question

Chunking	Retrieval	Rerank	P	R	F1	Ctx
SAC	Dense	Yes	0.280	1.000	0.417	549
SAC	Dense	No	0.280	1.000	0.417	574
SAC	Hybrid	Yes	0.280	1.000	0.417	549
SAC	Hybrid	No	0.260	0.967	0.392	575
Paragraph	Dense	Yes	0.200	1.000	0.333	1800
Paragraph	Dense	No	0.200	1.000	0.333	1508
Paragraph	Hybrid	Yes	0.200	1.000	0.333	1800
Paragraph	Hybrid	No	0.200	1.000	0.333	1552

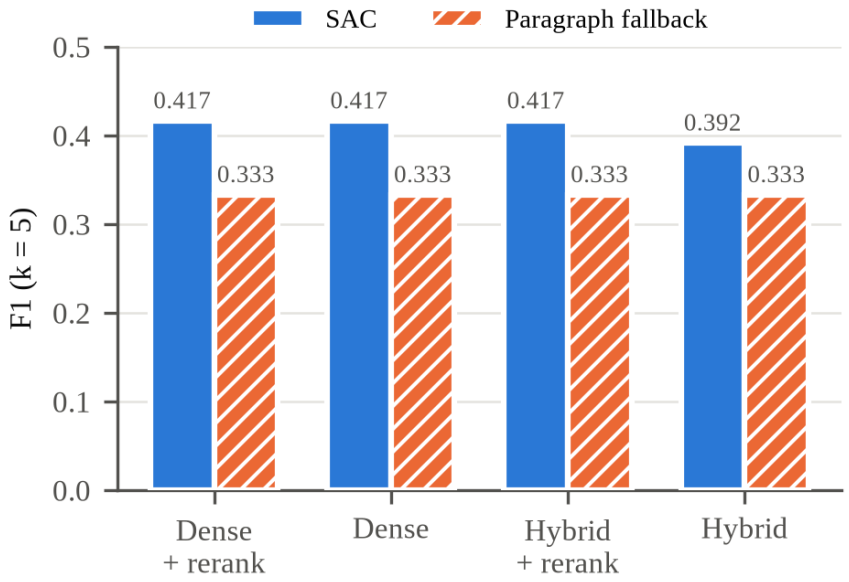


Fig. 6.1 Retrieval F1 for summary-augmented and paragraph chunking under each retrieval setting

The question set is too easy to separate the retrieval settings themselves. Every configuration found a gold clause for every question, so the retrieval rate was 100 percent everywhere, and dense and hybrid retrieval, with or without reranking, differed by at most one relevant chunk out of fifty. The reranker changed the result in only one configuration, summary-augmented chunking with hybrid retrieval, where it recovered one question and raised recall from 0.967 to 1.000. Precision across the two chunkers should also not be compared too directly, because each synthetic section is one short paragraph, so a paragraph chunk can hold all of a question's gold clauses, which caps its precision at 0.200 by construction. Reranking had a clear cost, adding 6.7 to 12.8 seconds per question on our hardware against under 0.1 seconds without it. We therefore draw no conclusion about the retrieval settings from these numbers alone.

6.4.2 Verification Ablation

To see what each verification layer catches, the cached answers to the ten questions were corrupted in two ways. In the first, every citation was replaced with a chunk identifier that was never retrieved, producing fourteen fabricated citations. In the second, each claim was replaced with a claim from a different question while its citation was kept, producing fourteen swapped claims. Together these give twenty-eight claims that must never be shown to the user. Table 6.3 removes one layer at a time and reports, for the clean answers, the mean score, the number of questions answered and refused and the number of weak claims shown, and, for the corrupted answers, the number of fabricated and swapped claims that reached the user. Fig. 6.2 shows the number of corrupted claims shown when each layer is removed, which makes the role of the gate visible at a glance.

Table 6.3 Verification Ablation: Clean Set of 10 Questions and 14 Claims; 14 Fabricated and 14 Swapped Claims

Arm	Mean VCS	Answered	Refused	Weak shown	Fabricated shown	Swapped shown
Full chain	0.768	7	3	0	0	0
Without V1	0.768	7	3	0	0	0
Without V2	0.813	8	2	1	0	0
Without V3	0.777	7	3	0	0	0
Without V4	0.723	7	3	0	0	0
Without V5	0.768	10	0	5	14	14
Without V6	0.768	7	3	0	0	0

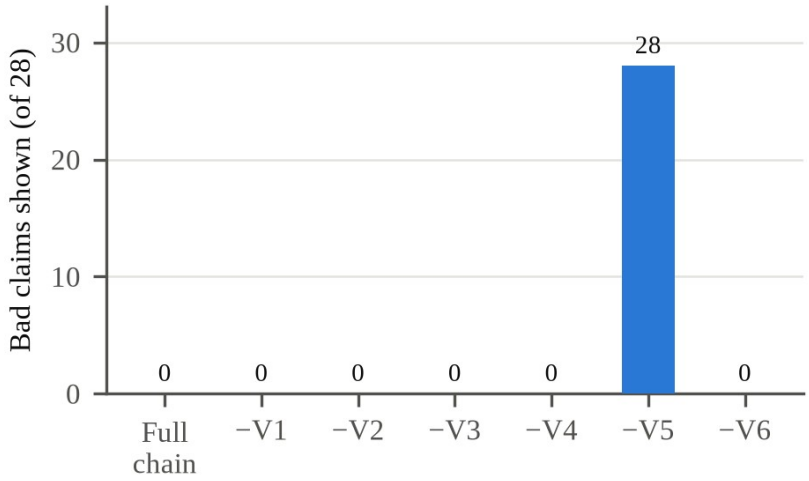


Fig. 6.2 Corrupted claims shown to the user when each layer is removed, out of 28; only removing the V5 gate lets them through

The full chain showed none of the twenty-eight corrupted claims, and neither did any arm with a single detection layer removed. Removing the gate, layer V5, showed all twenty-eight, and it also showed five clean claims that the full chain scores below 0.60. Without the gate, the detectors still scored every corrupted claim at zero, but a score changes nothing for the user unless something acts on it. The detectors also overlap: with V1 removed, a fabricated citation still scored zero, because V2 found no premise to test and the fidelity of V3 collapsed. The distinct contribution of V1 is diagnostic, since it is the only layer that names the missing identifiers in the proof. On clean input, V2 moved decisions the most; without it, one more question was answered, the mean score rose from 0.768 to 0.813 and one weak claim was shown. Removing V3 or V4 moved the mean score slightly but changed no decision, and V6 changes no score by design.

The ablation also shows the cost of caution. All ten questions retrieved their gold clause, yet the full chain answered only seven. In each of the three refusals, the inference model judged a correctly cited claim to be NEUTRAL rather than ENTAILS; one question about the grounds for eviction scored zero on all three of its claims without any hard gate firing. The same pattern appeared on real contracts, as described below. These false refusals are the clearest evidence that the threshold of 0.60 needs to be calibrated and that a general-purpose inference model is too strict for some legal wording. Even so, a false refusal is a safer failure than a false acceptance, because the user is told that the system could not verify an answer and can read the documents directly, whereas a wrong answer shown as verified could be acted upon without any warning at all.

6.4.3 End-to-End Test Through the Interface

The end-to-end test indexed five documents through the web interface: three non-disclosure agreements from ContractNLI, a non-competition agreement from CUAD and one of the synthetic statutes, which is the only one with section structure and therefore exercises summary-augmented chunking. Fifteen questions with answers fixed in advance were then asked, and Table 6.4 summarises the outcomes before and after fixing the defects found during the test. No wrong or mis-cited answer was ever shown as verified, and every verified claim was supported by the passage quoted in its proof. Two of the three unanswerable questions and both false-premise questions were refused, and the general concept questions were answered and labelled as general knowledge. The paraphrased question, which asked how much money a tenant is owed if thrown out by force, was answered correctly from the eviction clause with a score of 1.00.

Table 6.4 End-to-End Test: 15 Questions with Answers Fixed in Advance

Outcome	Before fixes	After fixes
Correct, shown as verified	3	4

Outcome	Before fixes	After fixes
Wrong, shown as verified	0	0
Correct abstention	4	5
False abstention	4	3
General knowledge, labelled	2	2
Other	2	1

The main failure was false abstention, which had four causes. Two were defects in our code: the model once wrote citations in full-width brackets, and a file name containing a comma split a citation identifier in two, and in both cases a correct answer was withheld. We fixed both, together with a routing error that sent a question about the tenancy act to the general-knowledge path, and added a regression test for each; re-checking the recorded answers with the fixed code gave a score of 1.00 for both, and asking again live verified one of them. The third cause was the inference model, which scored a correct claim about an appeal deadline at 0.51. The fourth was retrieval for comparison questions, discussed in the error analysis. The mean response time was 17.3 seconds per question, with a median of 16.1 seconds, almost all of it spent in the language model call.

6.4.4 LangChain and Plain-Python Comparison

The pipeline was first written in plain Python and later rebuilt with LangChain [33] for loading, retrieval and generation, while the structure parser and the verification chain remained our own code. To make sure the framework did not change the behaviour of the system, the two versions were compared on the same inputs, and Table 6.5 shows the results. The LangChain version returned identical top-five retrieval rankings on all ten labelled queries, with an F1 score of 0.417 and recall of 1.00 for both. Replaying the same cached model answers through both versions gave identical verification decisions and scores on all ten questions, and neither version surfaced any of the twenty-eight corrupted claims. The mean query latency was 13.86 seconds for plain Python and 14.50 seconds for LangChain, a difference of 4.6 percent that lies within the normal variation of the language model calls.

Table 6.5 Plain-Python and LangChain Implementations Compared

Measure	Plain Python	LangChain
Top-5 rankings identical on 10 queries	Reference	10 of 10
Retrieval F1 / recall	0.417 / 1.00	0.417 / 1.00
Verification decisions identical	Reference	10 of 10
Corrupted claims shown (of 28)	0	0
Mean query latency	13.86 s	14.50 s (+4.6%)

6.5 Comparison with Existing Methods

A direct numerical comparison with other legal question-answering systems was not possible, because they were evaluated on different documents and questions, and the commercial tools studied by Magesh et al. [7] are not available for testing on our data. The comparison is therefore made at the level of behaviour, using the design comparison of Chapter 4. A general chatbot gives no citations and no refusal, so every one of its errors reaches the user. A standard retrieval-augmented system gives citations but shows its answers without checking them, which corresponds to the ablation arm without the gate, in which all twenty-eight corrupted claims and five weak claims were shown. Systems with post-generation checks report problems, but without a decision rule they leave the decision to the user. Legal AI showed none of the corrupted claims and no wrong answer as verified in the end-to-end test, at the cost of refusing some correct answers.

6.6 Error Analysis

The errors observed fall into three groups. The first group is false abstention caused by the inference model. The roberta-large-mnli model was trained on general English and judges some correctly cited legal claims as NEUTRAL, for example when the cited clause does not repeat the name of a party that appears only in the heading of the agreement, or when a deadline is stated in a slightly different form. Because the entailment verdict carries the largest weight, such a verdict can pull an answer below the threshold. The second group is retrieval for comparison questions. A single candidate pool serves the whole question, so when a question asks about three agreements, the pool can be filled by two of them and the third never reaches the model. The third group, now fixed, consisted of parsing defects in citations and file names that turned correct answers into refusals.

The most instructive error combines retrieval and verification, and it is shown in Fig. 6.3. Asked for the governing law of three agreements, the system showed two correct and verified answers. The model's line saying that the third was not in the excerpts carried no citation, so the parser discarded it, and the user saw an answer that looked complete. Nothing false was shown, but verification checks what an answer says, not what it leaves out, so partial answers need to be detected and flagged. A related weakness comes from averaging the claim scores. In one answer, a verified score of 0.62 was the mean of two claims, the first contributing 1.00 and the second scoring below 0.25, and the weak claim was shown only because a strong claim sat next to it.

Gating each claim on its own score, or aggregating with the minimum instead of the mean, would have withheld it.

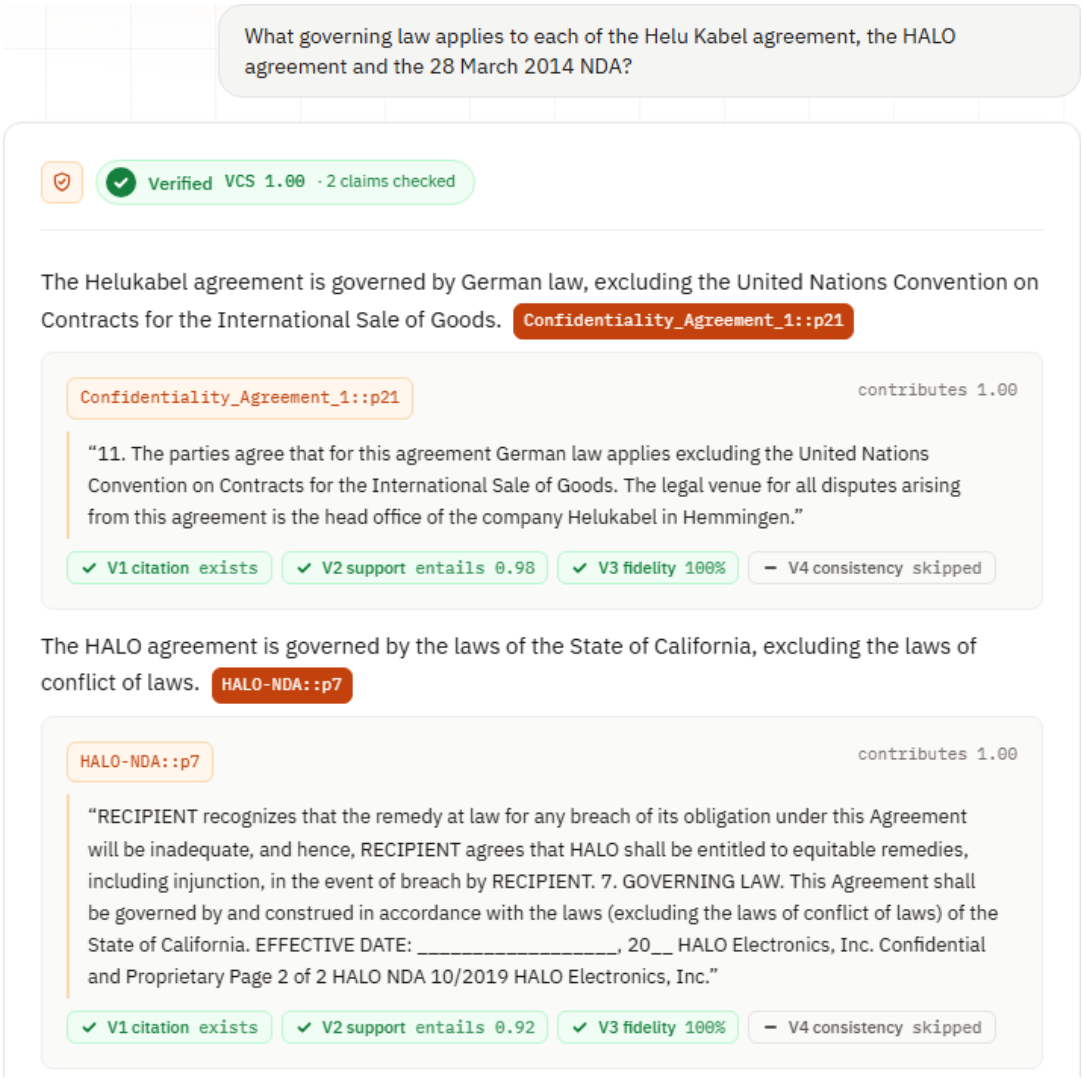


Fig. 6.3 A verified but incomplete answer: two correct claims for a question about three agreements; the note about the missing third had no citation and was dropped

6.7 Sample Outputs

Table 6.6 lists sample questions and the responses of the system, taken from the end-to-end test, the batch test on real contracts and the testing of the criminal law pack. They show the three kinds of response that the system can give. Questions answered from the documents carry a Verified badge, a score and a citation to a specific clause, as in the paraphrased tenancy question and the question about the parties to the Federal Express agreement, which were verified with scores of 1.00. A general question about the meaning of confidential information was answered from the documents with a score of 0.99, because the agreements define the term, while a question about what a non-disclosure agreement is was answered from general knowledge and labelled

accordingly. A question about repeat unfair trade practice offences was refused, because none of the indexed agreements covers it, as shown in Fig. 6.5.

Table 6.6 Sample System Outputs

Question	Response	Score	Evidence
If my landlord throws me out by force, how much am I owed?	Verified answer	1.00	Eviction clause, Section 7(a) of the tenancy act
Who are the parties to the Federal Express agreement?	Verified answer	1.00	Federal Express agreement
What is confidential information?	Verified answer	0.99	Definitions in the agreements
What is a non-disclosure agreement?	General knowledge, labelled	None	No citation by design
What penalty applies to a repeat unfair trade practice offence?	Abstained	None	Not covered by the indexed agreements



Fig. 6.4 Proof view of a verified answer on a real non-disclosure agreement, with the cited paragraph and the verdict of each layer

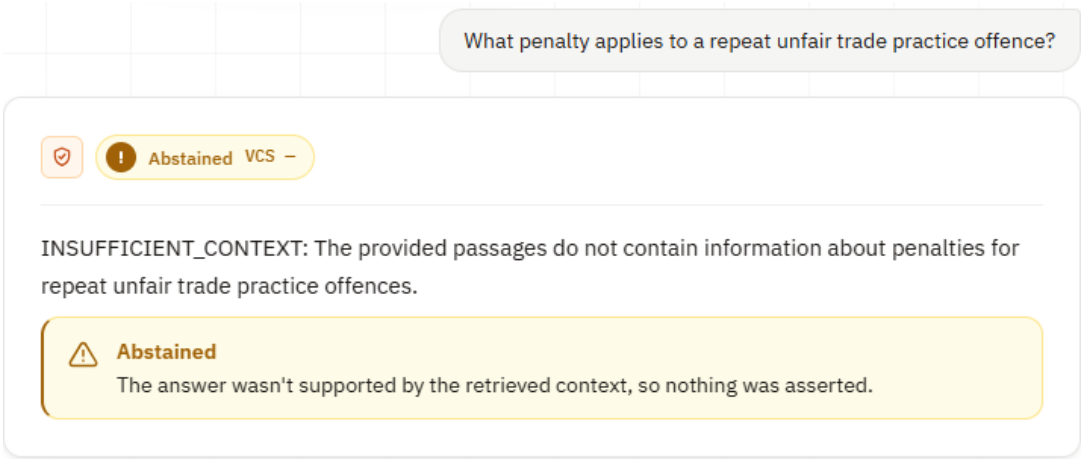


Fig. 6.5 An abstention: no indexed agreement covers repeat unfair trade practice offences, so no claim is shown

Fig. 6.4 shows the proof view of a verified answer on a real non-disclosure agreement. The first of two claims is open, showing the paragraph it cites and the verdict of each layer; layer V4 is shown as skipped because the interface does not resample answers, in order to keep the response time acceptable. Fig. 6.5 shows an abstention, in which the model returned the abstention marker and no claim was shown. Fig. 6.6 shows the Evaluation page's map of an index of 695 chunks from eight documents, projected to two dimensions with principal component analysis and coloured by document. The 412 chunks of the one document that is not an agreement form their own cluster, while the seven agreements overlap because they share most of their vocabulary. This overlap is the reason every chunk identifier names its source document, since a question can retrieve a similar clause from the wrong contract and the citation makes that visible.

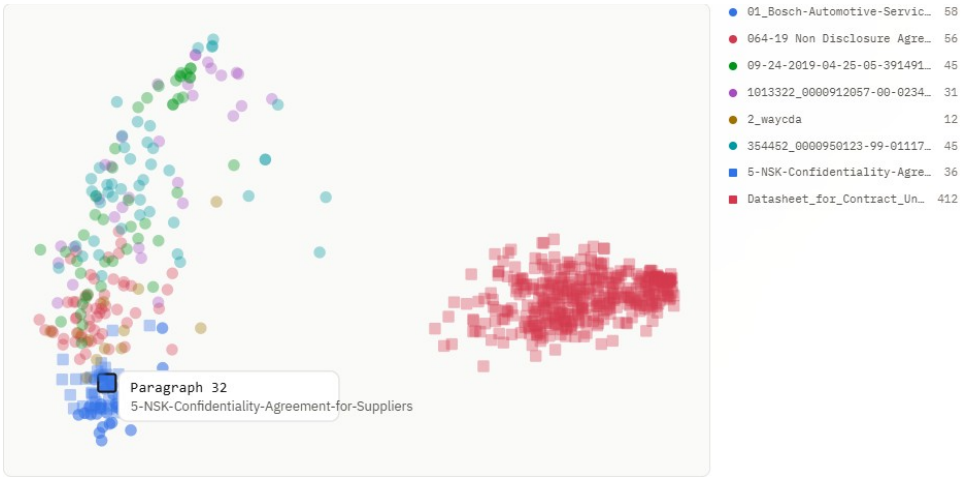


Fig. 6.6 The Evaluation page's map of the index: 695 chunks from eight documents, projected with PCA and coloured by document

6.8 Ethical and Responsible AI Considerations

A legal question-answering system can affect people's decisions about their rights and obligations, so responsible use was considered throughout the project. The most

important principle is that the system should not present unsupported information as reliable. This is why every answer passes through the verification chain, why an answer is withheld when its score is low, and why general-knowledge answers are labelled and never given citations or a score. The proof view lets a user check every claim against its source, which keeps the human in control of the final judgement. The system is presented as a tool for reading and understanding documents, not as a source of legal advice, and the demonstration pack is clearly marked as reflecting Indian criminal law as it stood before 1 July 2024, so that users do not mistake it for the current law.

Privacy and transparency were also considered. Uploaded documents and the index stay on the machine that runs the system, and the only data sent to an external service is the prompt containing the question and the retrieved passages, so users should not upload documents they are not permitted to share with the language model provider. The hosted version shares its conversation list with every visitor, which is acceptable for a demonstration but would need user accounts in a real deployment. All experiments, including the failures, are logged and reported, and every number in this report can be traced to a result file. Finally, the team used an AI assistant to help with parts of the coding and the writing of this report, as stated in the declaration; the design decisions, the experiments and the interpretation of the results are the team's own responsibility.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project set out to build a legal question-answering system whose answers can be trusted because they are checked before they are shown. Legal AI splits statutes along their legal structure with summary-augmented chunking, retrieves evidence with dense and keyword search merged by reciprocal rank fusion and reranked by a cross-encoder, forces the language model to cite a source passage for every claim, and verifies every claim with a six-layer chain that reduces citation, entailment, fidelity and consistency checks to a single Verification Confidence Score. The score decides whether the answer is shown or withheld, and every answer carries a proof that the user can open from its citations. The system is available as a web application with pages for uploading many documents at once, asking questions, inspecting retrieval and viewing evaluation results, and it can be run locally or hosted online.

The evaluation supports the main design decision. On the labelled question set, summary-augmented chunking raised retrieval F1 from 0.333 to 0.417 over paragraph chunking and passed about a third as much text to the generator with no loss of recall. Of twenty-eight deliberately corrupted claims, the full chain showed none; removing any single detection layer still showed none, while removing the gate showed all twenty-eight, which shows that the gate, not any single detector, is what keeps known-bad claims away from the user. In the end-to-end test through the interface, no wrong answer was shown as verified. The main cost of this caution is false abstention, in which a correct answer is refused, mostly because the general-purpose inference model judges some legal wording as neutral, and calibrating the gate is the main open problem identified by this work.

7.2 Limitations

The results should be read with several limitations in mind. The evaluation sets are small: ten questions with fourteen gold clauses over five synthetic statutes for the ablations, and fifteen questions over five documents for the end-to-end test, judged by one reviewer against answers fixed in advance but without blind judging. The retrieval benchmark is saturated, since every configuration found a gold clause for every question, so it cannot rank retrieval methods. The threshold of 0.60 is an uncalibrated default, and three of ten answerable questions were refused. The corrupted claims show what the chain catches, but not how often the model hallucinates on its own, which was not measured. The inference model was not trained on legal text, and its neutral verdicts

caused most false refusals, including claims that name a party that the cited clause does not repeat.

Some limitations are in the design itself. A single retrieval pool serves the whole question, so in comparison questions one document can be crowded out, and the verification chain cannot detect that part of an answer is missing. Averaging the claim scores lets a weak claim through when it sits next to a strong one. Reranking adds several seconds per question on a computer without a graphics processor, and free-tier rate limits of the language model service interrupted some batch runs. The demonstration pack of Indian criminal law reflects the Acts as they stood before 1 July 2024, and its clause letters number the parts of long sections in order rather than reproducing the official clause numbers. Finally, the hosted version shares its conversation history among visitors and uses the team's language model quota, which is suitable for a demonstration but not for public use.

7.3 Future Enhancements

Several improvements follow directly from the limitations. Comparison questions should retrieve evidence separately for each document they mention, and the system should show a partial-answer warning when the model reports that information is missing, so that incomplete answers are flagged rather than silently shortened. The title of the document should be added to the premise given to the inference model, as summary-augmented chunking already does for retrieval, so that a claim naming a party can be checked against a clause that does not repeat it. Each claim should be gated on its own score, or the claim scores should be aggregated with the minimum instead of the mean, so that one strong claim cannot carry a weak one. The threshold should be calibrated on a held-out set that includes unanswerable questions, so that correct refusals can be counted as well as false ones.

Beyond these corrections, the system should be evaluated on larger and harder benchmarks such as LegalBench-RAG [25] and on Indian statutes taken directly from India Code [31], which was the corpus this project originally set out to serve, including the new criminal laws that came into force in 2024. The inference model could be fine-tuned on legal text or replaced by a legally adapted model, in the spirit of LEGAL-BERT [11], to reduce neutral verdicts on correct citations. A question set with answers split across adjacent chunks would allow the neighbour-chunk expansion already implemented in the code to be measured. Support for HTML documents would allow the full ContractNLI release to be indexed, and user accounts with private document collections would make a public deployment possible. Support for questions in Indian languages would make the system useful to many more people.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, 2019.

[3] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, 2023.

[4] M. Dahl, V. Magesh, M. Suzgun and D. E. Ho, "Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models," Journal of Legal Analysis, vol. 16, no. 1, 2024.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[6] Mata v. Avianca, Inc., No. 22-cv-1461 (PKC), United States District Court for the Southern District of New York, 2023.

[7] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning and D. E. Ho, "Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools," Journal of Empirical Legal Studies, 2025.

[8] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of EMNLP, 2020.

[9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.

[10] M. Reuter, T. Lingenberg, R. Liepina, F. Lagioia, M. Lippi, G. Sartor, A. Passerini and B. Sayin, "Towards Reliable Retrieval in RAG Systems for Large Legal Datasets," in Proceedings of the Natural Legal Language Processing Workshop (NLLP), pp. 17–30, 2025.

[11] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras and I. Androutsopoulos, "LEGAL-BERT: The Muppets straight out of Law School," in Findings of EMNLP, 2020.

[12] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, 2009.

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP-IJCNLP, 2019.

[14] G. V. Cormack, C. L. A. Clarke and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proceedings of ACM SIGIR, 2009.

[15] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv:1901.04085, 2019.

[16] S. Xiao, Z. Liu, P. Zhang and N. Muennighoff, "C-Pack: Packaged Resources to Advance General Chinese Embedding," arXiv:2309.07597, 2023.

[17] S. R. Bowman, G. Angeli, C. Potts and C. D. Manning, "A Large Annotated Corpus for Learning Natural Language Inference," in Proceedings of EMNLP, 2015.

[18] A. Williams, N. Nangia and S. R. Bowman, "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference," in Proceedings of NAACL-HLT, 2018.

[19] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.

[20] S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer and H. Hajishirzi, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in Proceedings of EMNLP, 2023.

[21] P. Manakul, A. Liusie and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in Proceedings of EMNLP, 2023.

[22] A. Figueiredo, J. Lucas, T. Ribeiro et al., "Grounded in Law: A Multi-Stage Anti-Hallucination Pipeline for Legal RAG Systems in Brazilian Portuguese," in Proceedings of the 17th International Conference on Computational Processing of Portuguese (PROPOR), 2026.

[23] J. Maghakian, R. Sinha, M. Schettewi and G. Kaur, "Embedding-Free RAG," in Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 24974–24985, 2025.

[24] N. Guha, J. Nyarko, D. E. Ho, C. Ré et al., "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2023.

[25] N. Pipitone and G. H. Alami, "LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain," arXiv:2408.10343, 2024.

[26] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song and T. Zhang, "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models," in Proceedings of ACL, 2024.

[27] Y. Koreeda and C. D. Manning, "ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts," in Findings of EMNLP, 2021.

[28] D. Hendrycks, C. Burns, A. Chen and S. Ball, "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review," in NeurIPS Datasets and Benchmarks Track, 2021.

[29] S. H. Wang, A. Scardigli, L. Tang, W. Chen, D. Levkin, A. Chen, S. Ball, T. Woodside, O. Zhang and D. Hendrycks, "MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding," in Proceedings of EMNLP, 2023.

[30] civictech-India, Indian-Law-Penal-Code-Json [Data set], GitHub. Available: https://github.com/civictech-India/Indian-Law-Penal-Code-Json

[31] Legislative Department, Ministry of Law and Justice, Government of India, India Code: Digital Repository of All Central and State Acts. Available: https://www.indiacode.nic.in

[32] LanceDB Developers, LanceDB: Developer-friendly, serverless vector database for AI applications [Computer software]. Available: https://github.com/lancedb/lancedb

[33] H. Chase, LangChain [Computer software], 2022. Available: https://github.com/langchain-ai/langchain

[34] S. Ramírez, FastAPI [Computer software]. Available: https://github.com/fastapi/fastapi

[35] Meta Platforms, React: The library for web and native user interfaces [Computer software]. Available: https://react.dev

APPENDICES

Appendix A: Source Code and Repository

The complete source code of Legal AI, together with the data preparation scripts, the experiment configurations and result files, the documentation and the demonstration videos, is available in the public GitHub repository at https://github.com/vvvvvivekkk/LEGAL-AI-PROG. The repository contains the plain-Python pipeline in the src folder, the LangChain pipeline and the API used by the web application in the lc folder, the React web application in the web folder, the synthetic statutes, evaluation sets and Indian criminal law pack in the data folder, and the logged experiments in the experiments folder. The docs folder contains the architecture description, the notes written for each development phase, the research paper, a guide to the system, the list of demonstration questions and instructions for hosting. Every number reported in Chapter 6 can be traced to a result file in the experiments folder of this repository.

The history of the repository records how the system was built in phases: the scaffold and ingestion with summary-augmented chunking came first, followed by embedding and the LanceDB index, hybrid retrieval and reranking, citation-forced generation, the verification chain and proof object, the evaluation and ablation runners, and finally the web application, the LangChain version and the hosting configuration. Each phase has its own documentation in the docs folder, describing what was built, how it was tested and what was learned. The repository does not contain the index itself, the downloaded model weights or any API keys; these are created or supplied on the machine that runs the system. Anyone with the link can read the code, and the project can be cloned, installed and run by following the steps in Appendix B, so that the results can be reproduced by the department.

Appendix B: Running and Hosting the System

To run the system on a Windows computer, the repository is first cloned with Git. A Python 3.11 virtual environment named .venv-lc is created and the back-end packages are installed from the requirement file in the lc folder, and the web application's packages are installed with npm inside the web folder. The key for the language model service is placed in a file named .env at the root of the repository, which is never committed to the repository. After this one-time setup, double-clicking the start.bat file starts the back end on port 8000 and the web application on port 5173 in two windows and opens the browser. The first upload and the first question are slower than the rest, because the three local models are loaded into memory at that point, which takes between twenty and thirty seconds on an ordinary laptop computer.

The index starts empty on every new machine, because it is stored in a local folder that is not part of the repository. The twenty-seven files of the Indian criminal law pack can be loaded by selecting all of them together on the Ingest page, or by running the script scripts/ingest_folder.py while the back end is running, after which the Home page shows 27 documents and 493 chunks. To host the system online, the repository includes a development container configuration for GitHub Codespaces, which installs the packages, builds the web application, downloads the models and starts the server with the criminal law pack loaded on a public address, using the free monthly allowance of a personal GitHub account. A Dockerfile in the deploy folder packages the same application as one container for any host that provides about four gigabytes of memory, and the hosting guide in the docs folder explains both options step by step.

Appendix C: Demonstration Questions

Table C.1 lists a selection of the questions prepared for demonstrating the system with the Indian criminal law pack, together with the section that should be cited and the behaviour that is expected. The full list of ninety-one questions, grouped by topic and including easier variants and refusal tests, is kept in the file docs/questions.md in the repository, and the script scripts/check_demo_questions.py asks all of them through a running back end and reports the result for each one. The expected answers were checked against the text of the cited sections when the questions were written. They describe what a correct response should contain, not results measured for this report, and they are included here so that the reader can repeat the demonstration and compare the behaviour of the system with the expected behaviour for each question.

Table C.1 Selected Demonstration Questions and Expected Behaviour

Question	Expected source	Expected behaviour
What is the punishment for theft?	ipc_theft::s379	Verified: imprisonment up to three years, or fine, or both
What is the punishment for murder?	ipc_murder_and_culpable_homicide::s302	Verified: death or imprisonment for life, and fine
Can a child under seven years of age be punished for a crime?	ipc_general_exceptions::s82	Verified: no, nothing done by a child under seven is an offence
What is the punishment if a husband or his relatives treat a woman with cruelty?	ipc_marriage_offences_and_cruelty_498a::s498A	Verified: imprisonment up to three years and fine
Must the police tell a person why he is being arrested without a warrant?	crpc_arrest::s50	Verified: yes, the grounds of arrest must be communicated

Question	Expected source	Expected behaviour
What is the punishment for dishonour of a cheque for insufficient funds?	ni_act_cheque_bounce::s138	Verified: up to two years, or fine up to twice the cheque amount, or both
How many persons make an assembly an unlawful assembly?	ipc_unlawful_assembly_and_rioting::s141	Verified: five or more persons with a common object
If someone steals my mobile phone, how long can he be sent to jail?	ipc_theft::s379	Verified from a paraphrase: up to three years
Why is the punishment for theft ten years of imprisonment?	ipc_theft::s379	False premise: abstain or correct to three years
What is the fine for drunk driving?	Not in the pack	Abstain: the Motor Vehicles Act is not indexed
What is mens rea?	None	General knowledge, labelled, no citation